Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera

A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
Nesma Osama
Mazen Adel
Mohamed Khater

Supervised by

Dr. Salma Abdel Monem

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework that combines a high-level Python interface with native execution components. The system is organized around sex modules: Accelera Pipe, the Code Parallelizer, Auto Preprocessing, AutoML, Deployment, and the Benchmark Platform. Accelera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, and emits OpenMP-annotated C++ code. This report documents the design, methodology, and experimental results available for the implemented modules, while retaining the required report structure for the remaining modules.

Arabic Abstract

Acknowledgment

Table of Contents

List of Figures

List of Tables

List of Algorithms

List of Abbreviations

AI	Artificial Intelligence
AST	Abstract Syntax Tree
CPU	Central Processing Unit
ML	Machine Learning
OpenMP	Open Multi-Processing
RSS	Resident Set Size
SVC	Support Vector Classifier
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	**Email**	**Phone Number**
Mohamed Ashraf	mohamed.abdelaziz03@eng-st.cu.edu.eg	+2 01004538215
Nesma Osama	nesma.ahmed03@eng-st.cu.edu.eg	+2 01067193062
Mazen Adel	abc1@email.com	+2 01xxxxxxx
Mohamed Khater	abc1@email.com	+2 01xxxxxxx

Supervisor

Name	**Email**	**Number**
Salma Abdelmoniem	abc1@email.com	+2 01114302362

Chapter 1: Introduction

Accelera is a hybrid Python/C++ machine-learning framework designed to support the complete data-science workflow, starting from data preparation and model selection, and extending to graph-based execution, native code acceleration, and benchmark comparison. The project is not a single isolated algorithm. Instead, it is a collection of connected modules that cooperate to make machine-learning experiments easier to build, easier to repeat, and faster to execute when possible.

The main idea behind Accelera is simple: Python gives developers a productive environment for experimentation, but real machine-learning workflows often need more structure and better execution control. A typical workflow may include data cleaning, preprocessing, feature engineering, model selection, prediction, metric evaluation, reporting, and comparison between several alternatives. When these steps are written as separate scripts, repeated work, unclear execution order, and performance bottlenecks become common problems.

Accelera addresses this by combining five main units: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, `parallelizer`, and the benchmark platform. Table 1.1 gives a quick overview of these modules and how they contribute to the full system.

1.1 Motivation and Justification

Machine-learning development is often presented as a simple process of choosing a model and calling `fit`. In practice, the model is only one stage in a larger workflow. Before training, the data must be cleaned, split, transformed, encoded, scaled, and sometimes reduced. After training, the output must be evaluated, compared, reported, and possibly deployed or submitted to a benchmark system. This creates several practical problems.

- **Repeated preprocessing code:** Similar cleaning, encoding, scaling, and feature-selection steps are often rewritten for every dataset.
- **Manual model comparison:** Users may need to try many models and configurations before finding a good solution.
- **Linear workflow limitations:** Traditional scripts or simple pipelines may duplicate work when several branches share early operations.
- **Python loop bottlenecks:** Custom preprocessing functions written in Python may become slow when they contain large numeric loops.

Table 1.1: Main Accelera modules and their roles.

Module	Main role	Why it matters
`auto_preprocessing`	Prepares tabular, text, and image data using automated preprocessing steps.	Reduces repeated manual data-cleaning work and produces reusable preprocessing artifacts and reports.
`accelera_automl`	Searches for suitable classification and regression models under time and trial budgets.	Helps users compare model candidates systematically instead of manually testing each model.
`accelera_pipe`	Builds graph-based machine-learning workflows with preprocessing, model, prediction, metric, merge, and branch nodes.	Exposes workflow structure, supports branch execution, and allows fitted paths to be reused through an executed graph.
`parallelizer`	Analyzes supported Python/C++ loop-heavy code, inserts safe OpenMP pragmas, and compiles native callable extensions.	Targets custom preprocessing bottlenecks that would otherwise remain slow in pure Python.
Benchmark platform	Provides a prototype web platform for datasets, metrics, submissions, and leaderboard-style comparison.	Connects the framework to a future environment for fair pipeline evaluation and community comparison.

- **Weak experiment visibility:** Without reports, graphs, and saved artifacts, it becomes harder to understand what preprocessing and evaluation steps were applied.
- **Lack of fair comparison:** Pipeline results need a structured place where metrics, submissions, and benchmarks can be compared.

These problems justify the need for Accelera. The project provides automation where the workflow is repetitive, graph execution where the workflow contains branches, native acceleration where supported custom code is slow, and benchmark components where results must be compared. The motivation is not only speed; it is also organization, reproducibility, and usability.

The `auto_preprocessing` module is motivated by the fact that data preparation usually consumes a large part of the development effort. It handles common preprocessing decisions for tabular datasets, such as dropping duplicate records, detecting column types, imputing missing values, encoding categorical features, applying robust scaling, selecting features, saving preprocessors, and generating reports. It also supports text preprocessing using cleaning, tokenization, stemming, TF-IDF vectorization,

and target encoding. For image workflows, it provides preparation paths for classification tasks. This makes the project useful before the model-selection stage even begins.

The `accelera_automl` module is motivated by the difficulty of manual model selection. A user may not know which estimator, preprocessing choice, or hyperparameter setting will perform best. AutoML reduces this burden by evaluating candidate configurations under controlled budgets. This makes the search process more systematic and provides useful outputs such as the best model, score, leaderboard, and run history.

The `accelera_pipe` module is motivated by the need to represent machine-learning workflows as graphs rather than only as linear chains. In branch-heavy experiments, several alternatives may share the same preprocessing steps. A graph representation makes these dependencies explicit. It also allows the system to reason about execution order, branch selection, reuse of fitted paths, and intermediate-data lifetime. This gives the user a high-level Python interface while the backend manages a more structured execution model.

The `parallelizer` module is motivated by the performance gap between Python and native compiled code. Python is easy to write, but loop-heavy Python functions can be slow. The parallelizer attempts to optimize supported custom functions by converting them to C++, extracting loops, computing loop features, classifying safe parallelization opportunities, applying rule-based safety checks, inserting OpenMP pragmas, and compiling the result as a native Python-callable extension. If the optimization path fails, the original function is preserved, which keeps the workflow correct.

Finally, the benchmark platform is motivated by the need for organized comparison. A machine-learning framework becomes more useful when users can compare pipelines on common datasets and metrics. The benchmark module therefore represents a future-facing part of the project, where submissions, metrics, and leaderboard-style results can help users evaluate their solutions fairly.

1.2 Project Objectives and Problem Definition

The main objective of Accelera is to provide an integrated framework for constructing, preprocessing, executing, evaluating, optimizing, and comparing machine-learning workflows. The project aims to keep the familiar Python programming style while adding automation, graph execution, native acceleration, and benchmark-oriented evaluation.

The main objectives of the project are:

1. Build an automated preprocessing module for tabular, text, and image datasets.

2. Support data cleaning, missing-value handling, categorical encoding, robust scaling, TF-IDF text representation, image preparation, feature selection, and report generation.

Table 1.2: Problems addressed by Accelera.

Problem	**Accelera component**	**Expected benefit**
Manual and repetitive data preparation	`auto_preprocessing`	Automates common preprocessing and produces saved artifacts and reports.
Time-consuming model selection	`accelera_automl`	Searches candidate models and configurations under controlled budgets.
Repeated work in branch-heavy workflows	`accelera_pipe`	Represents the workflow as a graph and exposes shared execution structure.
Slow custom Python loops	`parallelizer`	Converts supported functions to native code and inserts safe OpenMP pragmas.
Difficult result comparison	Benchmark platform	Stores benchmark information, metrics, submissions, and leaderboard views.

3. Implement AutoML classifiers and regressors that search across supported model configurations using scoring metrics, cross-validation, time budgets, trial budgets, and optional ensemble construction.
4. Provide a graph-based pipeline module that represents workflow steps as executable graph nodes rather than only as a sequential list.
5. Support pipeline operations such as preprocessing, model fitting, prediction, metric evaluation, branching, merging, and executed-graph reuse.
6. Integrate custom preprocessing functions with the parallelizer so that supported loop-heavy functions can be optimized automatically.
7. Implement a conservative code parallelization path that uses C++, OpenMP, loop-feature extraction, classification, rule-based safety checks, and native compilation.
8. Provide reporting tools that make preprocessing outputs, model metrics, and pipeline structure more visible to the user.
9. Build a benchmark platform prototype for datasets, submissions, users, metrics, and leaderboard-based comparison.

The problem addressed by the project can be stated as follows:

> Machine-learning workflows in Python are easy to write but become difficult to organize, repeat, compare, and optimize when they include many preprocessing steps, model alternatives, custom functions, and evaluation paths. Accelera aims to preserve Python usability while adding automated preprocessing, budgeted model search, graph-based execution, safe loop-level acceleration, and benchmark-oriented evaluation.

This problem has two sides. The first side is **workflow complexity**. As the number of preprocessing steps, branches, and candidate models increases, the workflow becomes harder to manage manually. The second side is **execution performance**. Even if the workflow is correct, it may repeat shared work or spend too much time in slow Python loops. Accelera addresses both sides by combining high-level automation with lower-level execution support.

1.3 Project Outcomes

The project outcomes are the implemented modules and the integration between them. Each module provides a different part of the final system.

Outcome 1: Automated preprocessing

The `auto_preprocessing` module prepares datasets for machine-learning models. For tabular data, it handles duplicates, missing values, column-type detection, categorical encoding, robust scaling, target processing, feature selection, and report generation. For text data, it applies cleaning, tokenization, stemming, TF-IDF vectorization, and label encoding. For image data, it supports classification preparation. The outcome is a set of processed arrays, saved preprocessors, selected features, and reports.

Outcome 2: AutoML model search

The `accelera_automl` module provides estimator-style AutoML classes for classification and regression. It searches candidate configurations, evaluates them with cross-validation, supports warm-starting from dataset metafeatures, and can build voting or stacked ensembles. The outcome is a fitted final model, best score, leaderboard, and run history.

Outcome 3: Graph-based pipeline execution

The `accelera_pipe` module provides a graph-based pipeline interface. It represents preprocessing, model, prediction, metric, branch, and merge operations as graph nodes. The output of a pipeline run includes predictions and an `ExecutedGraph`, which stores the selected fitted path for later use. This outcome gives the project its workflow-execution layer.

Outcome 4: Source-level parallelization

The `parallelizer` module accelerates supported loop-heavy code. It can process C/C++ code, supported Python code, custom Python functions, and transformer methods. It extracts loops, computes features, classifies parallelization opportunities, applies safety rules, inserts OpenMP pragmas, and compiles native Python-callable extensions when possible. If any step fails, it falls back to the original Python implementation.

Outcome 5: Pipeline and parallelizer integration

One of the most important outcomes is the connection between `accelera_pipe` and the `parallelizer`. When a preprocessing node contains a supported custom Python function, the pipeline attempts to optimize it before adding it to the graph. This allows the system to combine graph-level workflow execution with code-level acceleration.

Outcome 6: Reporting and benchmark support

The project includes reports for preprocessing, model metrics, and graph visualization. It also includes a benchmark platform prototype with backend and frontend components for datasets, metrics, users, submissions, and leaderboards. These outcomes improve visibility and comparison.

Table 1.3: Summary of project outcomes.

Outcome	**Description**
Automated preprocessing	The project provides reusable preprocessing support for tabular, text, and image data, with generated reports and saved preprocessing artifacts.
Automated model selection	The AutoML module helps search and compare candidate classification and regression models under time and trial constraints.
Graph-based pipeline execution	Accelera Pipe represents machine-learning workflows as executable graphs, supporting preprocessing, modelling, prediction, metrics, branching, and merging.
Code-level acceleration	The Parallelizer attempts to optimize supported loop-heavy Python/C++ code using static analysis, safety checks, OpenMP pragmas, compilation, and caching.
Benchmarking support	The Benchmark Platform provides a foundation for dataset-based evaluation, prediction submission, metric calculation, and leaderboard-style comparison.

1.4 Document Organization

The rest of this report is organized into five main parts: Chapter 2 presents the market visibility study, Chapter 3 discusses the literature survey and related work, Chapter 4 explains the methodology and system design of the main Accelera modules, Chapter 5 presents testing, verification, and experimental results, and Chapter 6 concludes the report by summarizing the contributions, limitations, and possible future improvements. The appendices include supporting material such as documentation, usage instructions, feasibility details, and additional implementation notes.

Chapter 2: Market Visibility Study

This chapter studies the market need for Accelera and compares it with existing tools that solve related problems. The goal is to show who can benefit from the project, what alternatives already exist, and why Accelera provides a different value. The chapter also presents a simplified business case and financial analysis to clarify how the project could be developed into a product or service.

Accelera targets the data-science workflow as a complete process, not as one isolated task. Some existing tools focus on preprocessing, others focus on model selection, and others focus on competition platforms or code acceleration. Accelera combines these directions through five connected parts: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, `parallelizer`, and the benchmark platform.

2.1 Targeted Customers

The targeted customers for Accelera are users and organizations that build, test, compare, and optimize machine-learning workflows. These customers can be divided into several groups, as shown in Table 2.1.

The most important customer segment is the data-science and machine-learning user who wants to move from a raw dataset to an evaluated model with less manual work. For beginners, Accelera provides guidance and automation. For experienced users, it provides a more structured execution model and opportunities for acceleration.

Accelera can also be useful in educational environments. In many machine learning courses, students repeat the same steps: clean the data, encode features, scale numerical values, split the dataset, train models, calculate metrics, and compare results. Accelera can make these steps easier to understand because it provides reports and separates the workflow into clear modules.

For professional users, the value is different. They care more about reproducibility, execution time, reusable pipeline structure, and fair comparison. The `accelera_pipe` and benchmark modules are especially useful here because they organize workflow execution and comparison. The `parallelizer` adds value when the workflow includes supported custom preprocessing functions that are slow in Python.

Customer Value Proposition

The main value proposition of Accelera is that it reduces the gap between experimentation and structured execution. Instead of forcing the user to choose between simple scripts and large industrial platforms, Accelera provides a research-friendly framework with practical automation and optimization features.

Table 2.1: Targeted customers for Accelera.

Customer group	Main need	How Accelera helps
Students and beginners	Need an easier way to build complete machine-learning workflows without writing every step manually.	Provides automated preprocessing, model search, reports, and a pipeline interface that organizes the workflow.
Data scientists	Need to prepare data, compare models, and repeat experiments quickly.	Reduces repetitive preprocessing, supports AutoML search, and gives reports and leaderboards for comparison.
Machine-learning engineers	Need structured, reusable, and optimized workflows that can be inspected and executed reliably.	Provides graph-based pipeline execution, executed-graph reuse, and integration with custom preprocessing acceleration.
Researchers	Need to compare approaches, measure results, and experiment with workflow and code optimization.	Provides modular components for benchmarking, graph execution, OpenMP parallelization, and performance evaluation.
Small teams and startups	Need a practical toolkit that reduces development time without building an internal ML platform from scratch.	Combines preprocessing, AutoML, pipeline execution, reports, and benchmark support in one project.

- **Less manual work:** preprocessing and model search are partially automated.
- **Better organization:** workflows can be represented as graph pipelines instead of scattered scripts.
- **Better visibility:** reports and benchmark views help users inspect results.
- **Possible speedup:** supported loop-heavy preprocessing code can be accelerated through C++ and OpenMP.
- **Fairer comparison:** benchmark components help compare pipelines using common datasets and metrics.

2.2 Market Survey

Many existing tools provide features related to Accelera, but most of them focus on only one part of the workflow. Some tools focus on preprocessing, some on AutoML, some on competitions, and some on code acceleration. Accelera is different because it tries to connect these ideas into one toolkit.

Table 2.2: Market comparison summary.

Tool or platform	Main focus	Limitation	Accelera difference
`scikit-learn`	Machine-learning models, preprocessing tools, and linear pipelines.	Pipeline execution is mainly linear and does not focus on graph-based branch execution or custom code parallelization.	Adds graph-based pipeline execution and integration with the Parallelizer.
AutoCleanML	Automatic data cleaning and preprocessing.	Focused mainly on data cleaning and does not cover the whole workflow.	Adds AutoML, graph pipelines, parallelization, reports, and benchmarks.
Auto-sklearn / TPOT / AutoGluon	Automated model selection and machine-learning search.	Focus mainly on model search, not graph execution or source-level code parallelization.	Combines AutoML with preprocessing, graph pipelines, and the Parallelizer.
Numba / Cython	Python performance acceleration.	Require user knowledge of acceleration syntax and do not manage ML workflow structure.	Provides an automatic attempt to optimize supported preprocessing functions inside the pipeline.

2.2.1 AutoCleanML

AutoCleanML is a Python tool focused on automating data cleaning and preprocessing tasks for machine-learning datasets. It is useful because data preparation is one of the most repetitive and time-consuming parts of the machine-learning workflow.

Pros:

- Automates common data-cleaning and preprocessing decisions.
- Helps beginners prepare datasets more easily.
- Provides reporting that explains some preprocessing decisions.

Cons:

- Focuses mainly on preprocessing, not the complete workflow.
- Does not provide a graph-based pipeline execution engine.

- Does not target custom code acceleration or OpenMP parallelization.
- Does not provide a benchmark platform for comparing full pipelines.

Compared with AutoCleanML, Accelera includes preprocessing but does not stop there. It also includes AutoML search, graph pipeline execution, source-level parallelization, reports, and benchmark support. Therefore, Accelera aims to be closer to a complete workflow framework.

2.2.2 `scikit-learn`

`scikit-learn` is one of the strongest tools in the machine-learning market. It provides a wide range of models, preprocessing tools, metrics, and a simple `Pipeline` abstraction. For this reason, it is the most important competitor for `accelera_pipe`.

Pros:

- Mature and widely used.
- Provides many models, preprocessing utilities, and metrics.
- Has a simple and reliable pipeline interface.
- Integrates well with the Python data-science ecosystem.

Cons:

- The standard pipeline is mainly linear.
- Branch-heavy workflows may require repeated pipelines or external search utilities.
- It does not focus on native graph scheduling for pipeline branches.
- It does not automatically convert supported custom preprocessing functions to C++ and OpenMP.

Accelera uses `scikit-learn` as a strong baseline rather than ignoring it. The difference is that `accelera_pipe` represents the workflow as a graph and integrates with the Parallelizer for supported custom preprocessing acceleration.

2.2.3 AutoML Tools

AutoML tools such as Auto-sklearn, TPOT, and AutoGluon focus on automating model selection and hyperparameter search. They are useful because they reduce the manual effort needed to test many models.

Pros:

- Search over many candidate models and configurations.

- Reduce manual model-selection effort.
- Often provide strong baseline performance.
- Some tools support ensembles.

Cons:

- Usually focus on model search more than workflow execution.
- Do not mainly target graph-based pipeline execution.
- Do not automatically accelerate custom preprocessing loops using OpenMP.
- Can be heavy or complex for beginners.

Accelera includes `accelera_automl`, but also connects AutoML with preprocessing, graph execution, reports, and parallelization. This makes the AutoML module one part of a larger framework rather than the only feature.

2.2.4 Code Acceleration Tools

Tools such as Numba, Cython, and manual C++ extensions can speed up Python code. They are relevant because Accelera also tries to accelerate supported custom preprocessing functions.

Pros:

- Can significantly improve performance for numeric code.
- Useful for loop-heavy workloads.
- Provide more control for experienced developers.

Cons:

- Often require users to understand extra syntax, compilation, or type constraints.
- Do not automatically decide where acceleration fits inside an ML pipeline.
- Do not provide pipeline graphs, AutoML, reports, or benchmark features.

The Accelera Parallelizer is different because it is designed to work with the pipeline module. It attempts to optimize supported custom preprocessing functions and attach the optimized callable back to the pipeline node. This makes code acceleration part of the workflow instead of a separate manual task.

2.3 Business Case and Financial Analysis

Accelera can be developed as an open-source framework with optional hosted services around benchmarking, reporting, team collaboration, and managed execution. The open-source part helps adoption because students, researchers, and developers can use the framework freely. The business value can come from hosted benchmark features, team dashboards, managed experiment tracking, and support services.

2.3.1 Business Case

The main business case is based on reducing time spent in repetitive data-science work. Users and teams spend time preparing data, comparing models, rewriting scripts, debugging pipeline steps, and measuring results. Accelera can reduce this time by providing one framework that handles multiple stages of the workflow.

Table 2.3: Possible business value of Accelera.

Business value	**Reason**	**Possible customer**
Faster experimentation	Automated preprocessing and AutoML reduce repeated manual trials.	Students, data scientists, small teams.
Better organization	Graph pipelines and executed graphs make workflows easier to inspect and reuse.	ML engineers and research teams.
Performance improvement	Parallelizer can speed up supported loop-heavy preprocessing code.	Teams with custom preprocessing bottlenecks.
Better comparison	Benchmark platform supports submissions, metrics, and leaderboards.	Universities, communities, and organizations.
Lower entry barrier	Reports and automation make ML workflows easier for beginners.	Education and training programs.

The first possible product direction is a hosted benchmark platform. Users can submit pipelines, compare results on public datasets, and view leaderboards. A free tier can support public datasets and basic submissions, while paid tiers can support private benchmarks, team spaces, larger storage, and advanced reports.

The second possible product direction is a managed workflow dashboard. This dashboard can visualize preprocessing results, model leaderboards, pipeline graphs, and runtime comparisons. Teams can use it to organize experiments and share results without relying only on notebooks.

The third possible product direction is professional support and integration. Orga-

nizations that want to use Accelera internally may need help adapting the framework to their datasets, infrastructure, or benchmark requirements. This creates a possible support and consulting model.

2.3.2 Financial Analysis

The financial analysis can be divided into expected costs and possible revenue streams. Since Accelera is mainly a software project, the capital expenditure is limited compared with hardware-based products. Most costs come from development, hosting, cloud resources, storage, and maintenance.

Table 2.4: Expected cost categories.

Cost category	Description	Notes
Development cost	Time spent implementing, testing, documenting, and maintaining the framework.	Main cost during early stages.
Cloud hosting	Servers for benchmark backend, database, frontend, and possible job execution.	Can start small and scale with users.
Storage	Datasets, submissions, reports, and model artifacts.	Increases with benchmark usage.
Compute resources	Running benchmark submissions or managed experiments.	Can be limited by quotas or paid tiers.
Maintenance and support	Bug fixes, user support, documentation, and security updates.	Needed for long-term usage.

Possible revenue streams include:

- **Free open-source adoption:** increases user base and community trust.
- **Hosted benchmark subscriptions:** paid private benchmarks, team workspaces, and larger storage limits.
- **Managed experiment dashboards:** paid reporting and workflow monitoring for teams.
- **Enterprise support:** integration, customization, and technical support for organizations.
- **Education packages:** university or training-center packages for teaching machine-learning workflows.

A simple financial plan can start with low-cost hosting and manual support. In the first stage, the focus should be adoption and validation. The open-source framework can attract users, while the benchmark platform can collect early feedback. In the second stage, private benchmark spaces and team dashboards can be introduced. In the third stage, enterprise support and managed compute can be added if the user base grows.

Table 2.5: Simplified business plan stages.

Stage	**Focus**	**Expected result**
Stage 1	Release open-source framework, documentation, examples, and local benchmark prototype.	Build trust, collect users, and validate the technical direction.
Stage 2	Launch hosted benchmark features, public leaderboards, and private team workspaces.	Create first revenue stream through hosted services.
Stage 3	Add managed experiment dashboards, larger storage, and paid support.	Serve teams and organizations with more advanced workflow needs.
Stage 4	Provide enterprise integration and managed compute for benchmark execution.	Scale the platform into a commercial ML workflow service.

The expected break-even point depends on hosting cost, development effort, and the number of paying users. Since the software can begin as open source, the initial financial risk is lower than products that require manufacturing or special hardware. The main challenge is user adoption. If Accelera can attract students, researchers, and small teams through the free framework, the benchmark and reporting services can become the commercial layer that supports future development.

Chapter 3: Literature Survey

In this chapter, we give an overview of the topics, algorithms, and techniques needed to understand the project as a whole. We discuss the technical subjects and background knowledge, and then summarize the research papers related to our project.

3.1 Background and Previous Work

This section introduces the main background needed to understand the Accelera project as a whole. It explains the important concepts, techniques, and tools that were used in different parts of the system. These background topics help clarify how the project supports data preparation, pipeline execution, report generation, model evaluation, and benchmarking. The goal of this section is to give the reader enough context before moving to the methodology and implementation chapters.

3.1.1 Data Science and Machine Learning

Data science is the field that uses data, statistics, programming, and domain knowledge to understand a problem and extract useful insights from it. It is usually used when we have raw data and we want to turn it into useful information or predictions.
Machine learning is one of the main parts of data science. Instead of writing fixed rules for every case, we train a model on examples, and the model learns patterns from these examples. After training, the model can use what it learned to make predictions on new data.
In supervised machine learning, the training data contains input features and a target label. The model learns the relationship between the features and the target. If the target is a class, such as spam or not spam, the problem is called classification. If the target is a continuous number, such as price or temperature, the problem is called regression.

3.1.1.1 Data Exploration and Preprocessing [1]

Exploratory Data Analysis, or EDA, is the step where we inspect the dataset before building the model. We summarize columns, draw visualizations, check missing values, and try to understand relationships between variables. This step helps us notice problems early instead of discovering them after training the model.
Data preprocessing is the stage of transforming raw data into a clean and usable format for machine learning models. In real datasets, the data may contain missing values, duplicated rows, different feature scales, and categorical columns that cannot be used directly by many models. Because of this, preprocessing is usually considered an essential part of a machine learning workflow.

Missing value imputation is used when some values in a dataset are empty. Instead of removing all rows that contain missing values, the missing values can be replaced using a suitable value, such as the mean, median, mode, or another estimated value. Robust scaling is a scaling technique that uses the median and the interquartile range instead of the mean and standard deviation. It is useful when the data contains outliers, because the median and interquartile range are less affected by extreme values:

$$x' = \frac{x - \text{median}(X)}{\text{IQR}(X)}$$

where x' is the scaled value, x is the original value, $\text{median}(X)$ is the median of the feature values, and $\text{IQR}(X)$ is the interquartile range.
Categorical encoding is the process of converting text categories into numeric values. Many traditional machine learning models and preprocessing pipelines expect numerical input. Two examples are binary encoding and frequency encoding. Binary encoding first gives each category a number, then converts this number into binary form and stores the binary digits in separate columns. Frequency encoding replaces each category with how often it appears in the dataset:

$$f(c) = \frac{\text{count}(c)}{N}$$

where $f(c)$ is the frequency of category c, $\text{count}(c)$ is the number of rows containing this category, and N is the total number of rows.

3.1.1.2 Model Evaluation Metrics [2]

After training a model, we need metrics to check whether the model is performing well or not. The selected metric depends on the type of problem. Classification models are evaluated using metrics that compare predicted classes with actual classes, while regression models are evaluated using metrics that measure the difference between predicted numerical values and real values.

Classification metrics: Classification metrics are used when the output is a class or category, such as spam or not spam, positive or negative, or one image class from many classes. The confusion matrix is the base idea behind many classification metrics. It compares the actual classes with the predicted classes. For binary classification, it contains four values: true positives, true negatives, false positives, and false negatives.

- **True Positive (TP):** The model predicts positive, and the actual class is positive.
- **True Negative (TN):** The model predicts negative, and the actual class is negative.
- **False Positive (FP):** The model predicts positive, but the actual class is negative.

- **False Negative (FN):** The model predicts negative, but the actual class is positive.

Accuracy measures how many predictions are correct out of all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives. Accuracy is simple and useful when the classes are balanced, but it may be misleading when one class is much larger than the other.
Precision measures how many predicted positive samples are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

Precision is useful when false positives should be reduced.

Recall measures how many actual positive samples were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

Recall is useful when false negatives should be reduced.

The F1-score combines precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It is useful when both precision and recall are important, especially with imbalanced datasets.
Regression metrics: Regression metrics are used when the output is a numerical value, such as price, score, or temperature.
Mean Absolute Error measures the average absolute difference between the real values and predicted values:

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of samples. MAE is easy to understand because it gives the average error in the same unit as the target value.

Mean Squared Error gives more penalty to large errors:

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

This means large mistakes affect the score more than small mistakes.

Root Mean Squared Error is the square root of MSE:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE is easier to read than MSE because it returns the error to the same unit as the target value.

R-squared score measures how well the regression model explains the variation in the actual values:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and $\bar{y}$ is the mean of the actual values. A value close to 1 means the model explains the data well, while a value close to 0 means the model does not explain the data well.

3.1.2 Natural Language Processing (NLP) [3]

Natural Language Processing is the field that helps computers understand and work with human language. It is used in many applications such as text classification, sentiment analysis, spam detection, and chatbots. Text data cannot be used directly by most machine learning models, so it needs to be cleaned and converted into numbers first.
In text preprocessing, the text may be converted to lowercase, cleaned from unnecessary symbols, split into words, and simplified using stemming. After that, methods such as TF-IDF are used to transform the text into numerical features. In Accelera, these steps are used to prepare text datasets so they can be used for machine learning models.

3.1.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency is one common technique used to convert text into numerical features. It gives each word a weight based on how important it is in one document and across the full dataset. Term Frequency measures how often a word appears in a document. Inverse Document Frequency gives lower weight to words that appear in many documents, because these words are usually less useful for classification:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TFIDF(t, d)$ is the weight of term t in document d, $TF(t, d)$ is the frequency of the term in the document, and $IDF(t)$ measures how rare the term is across all documents.

3.1.3 Data Augmentation

Data augmentation is a technique used to increase the diversity of a dataset without actually collecting new data. It works by applying transformations to existing samples to create new modified versions. For example, in image classification, augmentation may include flipping, rotating, cropping, or changing brightness. This can help the model generalize better because it sees more variations during training.

3.1.4 Machine Learning Pipelines

A machine learning pipeline is a sequence of steps used to prepare data, train a model, make predictions, and evaluate results. Pipelines are useful because they organize the full workflow in one structure instead of writing each step separately.

A typical pipeline may include preprocessing steps such as scaling, encoding, or feature selection, followed by a machine learning model. This makes experiments easier to repeat and reduces mistakes, because the same preprocessing steps are applied during training and testing.

One of the most common tools for building machine learning pipelines is `scikit-learn`. It provides a `Pipeline` object that connects transformers and estimators together. In this project, `scikit-learn` is considered the main competitor for the `accelera_pipe` module because both systems aim to help users build machine learning workflows in Python.

3.1.5 Graph-Based Pipeline Execution

Traditional machine learning pipelines usually execute steps in a linear order. This is suitable for simple workflows, but it can become inefficient when several branches share the same preprocessing steps or when multiple models are compared.

Graph-based execution represents the workflow as nodes and edges. Each node represents an operation, such as preprocessing, model training, prediction, or metric evaluation. Each edge represents the dependency between operations. This representation can help avoid repeated work, because shared steps can be executed once and reused by different branches.

The `accelera_pipe` module follows this idea by representing the pipeline internally as an execution graph. This allows the system to support branching, merging, parallel execution of independent paths, and better management of intermediate data.

3.1.6 Code Parallelization

Code parallelization is the process of running parts of a program at the same time to reduce execution time. In many programs, the most expensive part is a loop that repeats the same operation over a large amount of data. If the loop iterations are independent, they may be executed in parallel.

Manual parallelization is difficult because the developer must check that there are

no unsafe dependencies between iterations. If a loop writes to shared variables or depends on results from previous iterations, parallel execution may produce wrong results. Because of this, automatic parallelization tools try to analyze code and decide whether a loop can be safely parallelized.

3.1.7 OpenMP

OpenMP is a programming interface used to add parallel execution to C and C++ programs. It allows developers to use compiler directives, called pragmas, to tell the compiler that some parts of the code can run in parallel.

For example, the directive `#pragma omp parallel for` can be used before a loop to distribute loop iterations across multiple threads. OpenMP is useful because it does not require rewriting the whole program. However, the pragma must be inserted only when the loop is safe to parallelize.

3.2 Comparative Study of Previous Work

3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [4]

Koukaras and Tjortjis discuss the role of data preprocessing and feature engineering in machine learning and data mining projects. They explain that preprocessing is not only a preparation step before model training, but also an important part of the complete machine learning pipeline. The paper also presents a general preprocessing framework that includes data cleaning, data transformation, feature construction, feature selection, and dimensionality reduction. For each stage, it explains different techniques and discusses the advantages and disadvantages of each one.

3.2.2 `scikit-learn` Pipeline

`scikit-learn` is one of the most widely used machine learning libraries in Python [9]. It provides many preprocessing tools, machine learning models, evaluation metrics, and utilities for building complete machine learning workflows.

The `Pipeline` object in `scikit-learn` allows users to connect several processing steps together. Each step is usually a transformer, and the final step is usually an estimator. This makes the workflow cleaner and easier to reuse.

In this project, `scikit-learn` is the main competitor for `accelera_pipe`. Both systems provide a way to build machine learning workflows. However, `scikit-learn` pipelines are mainly linear, while `accelera_pipe` uses a graph-based backend. This allows `accelera_pipe` to support branching, shared intermediate results, and parallel execution of independent parts of the workflow.

3.2.3 OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation

OMPar is a related work that focuses on automatic parallelization using AI-driven source-to-source compilation [10]. The main idea of OMPar is to analyze C and C++ code, detect loops that can be parallelized, and generate OpenMP pragmas automatically.

OMPar is important for this project because it inspired the general direction of the Parallelizer module. Both approaches use a staged process that includes loop analysis, loop classification, and pragma insertion. This makes the problem easier to handle because the system does not treat parallelization as one single step.

However, there is an important difference between OMPar and our Parallelizer. OMPar uses an AI-based pragma generation approach. In our project, the pragma generator was replaced with a rule-based generator. This decision was made because

the available training data was limited, and some of the available examples were synthetic data. Depending completely on generated or biased data could lead to unsafe pragma generation.

Therefore, our Parallelizer uses AI mainly to support loop classification, while the final pragma insertion is controlled by deterministic safety rules. This makes the system more conservative, but also safer and easier to explain.

3.2.4 Automated Machine Learning

Automated machine learning treats model-family selection, preprocessing choices, and hyperparameter tuning as one optimization problem. Instead of manually testing a small set of estimators, an AutoML system builds a search space and evaluates configurations under a consistent validation protocol. The combined algorithm selection and hyperparameter optimization problem can be written as

$$(a^*, \lambda^*) = \arg \min_{a \in \mathcal{A},\, \lambda \in \Lambda_a} \widehat{L}(a, \lambda; D),$$

where $\mathcal{A}$ is the set of candidate algorithms, Λ_a is the hyperparameter domain of algorithm a, and $\widehat{L}$ is the validation loss on dataset D. This formulation was popularized by Auto-WEKA [5].

3.2.5 Search Strategies for AutoML

Grid search evaluates a fixed Cartesian product and becomes expensive as the number of dimensions increases. Random search is often stronger when only some hyperparameters strongly affect performance [13]. Sequential model-based optimization improves on random sampling by fitting a surrogate model to observed configuration costs. SMAC uses random forests for algorithm configuration and acquisition-function based candidate selection [12].

3.2.6 Multi-Fidelity Evaluation and Warm Starts

Full cross-validation on the complete dataset is costly, especially for ensembles and iterative models. Multi-fidelity optimization first evaluates candidates using cheaper approximations, such as smaller samples, fewer folds, or reduced model budgets. Poor candidates can be discarded early, while promising candidates are promoted to more expensive stages.

Meta-learning is another way to reduce cold-start cost. A new dataset can be described using metafeatures, compared with historical datasets, and initialized with configurations that worked well on similar tasks. Auto-sklearn combines Bayesian optimization, dataset metafeatures, and ensemble construction for Scikit-learn workflows [6].

3.2.7 Ensemble Learning in AutoML

Ensembles reduce the risk of selecting one unstable model. Voting combines predictions or probabilities from several models. Stacked generalization trains a meta-model on out-of-fold predictions from base models [14]. The out-of-fold step is important because it prevents the meta-model from learning on predictions made by base models on their own training rows.

Table 3.1: Selected AutoML systems and their design emphasis.

System	Design emphasis
Auto-WEKA	Combined algorithm selection and hyperparameter optimization over WEKA components [5].
Auto-sklearn	Bayesian optimization, dataset metafeatures, and post-hoc ensembles for Scikit-learn workflows [6].
TPOT	Evolutionary search over tree-structured machine-learning pipelines [7].
FLAML	Cost-aware lightweight AutoML for limited resource budgets [8].
AutoGluon Tabular	Multi-layer stacking and ensembling with a high-level tabular prediction interface [11].
Accelera AutoML	Conditional model spaces, staged fidelity, surrogate-guided sampling, packaged warm starts, and voting or stacking inside Accelera.

3.3 Implemented Approach

3.3.1 Auto Preprocessing

In this module we decided to implement an automated preprocessing approach for tabular datasets. The approach was designed to prepare data for machine learning models by applying suitable preprocessing steps automatically according to the type and characteristics of each column and some user configurations. The chosen approach follows a classical automated preprocessing pipeline. The main preprocessing decisions are:

- **Missing values:** Median imputation is used for numerical features because it is simple and more robust to outliers than mean imputation. For categorical features, most-frequent imputation is used because it replaces missing values with valid existing categories.
- **Numerical scaling:** `RobustScaler` is used for numerical features because tabular datasets may contain outliers or skewed values. It depends on the median

and interquartile range, which makes it more stable than standard scaling when extreme values exist.

- **Categorical encoding:** The encoding method is selected according to the feature type and cardinality. Binary columns are encoded using ordinal encoding, low-cardinality categorical columns are encoded using binary encoding, and high-cardinality categorical columns are encoded using frequency encoding. This helps reduce the number of generated features and avoids high dimensionality.
- **Feature selection:** Mutual information is used because it is a filter-based method that can work with both classification and regression tasks. It helps keep the most informative features and remove weak features before model training.

This approach was chosen because it gives a good balance between automation, simplicity, and speed. More advanced techniques, such as K-nearest neighbour imputation, autoencoders, and wrapper feature selection, can be useful in specific cases, but they need more computation.
One of the important additions in this project is that the module is not only used for data preparation. It also supports exploratory data analysis report.
Also the module was extended to support text and image data preparation, not only tabular data.

3.3.2 `accelera_pipe`

In this module, we decided to implement a graph-based machine learning pipeline approach. The goal of this module is to make machine learning workflows easier to build while also improving execution when the workflow contains branches or repeated shared steps.

The general framework of the approach is as follows:

1. The user builds a pipeline using high-level Python methods.
2. The pipeline stores each operation as a node in an internal graph.
3. Preprocessing, model training, prediction, metric evaluation, branching, and merging are represented as graph operations.
4. The graph executor determines the correct order of execution.
5. Independent branches can be executed in parallel when possible.
6. Shared intermediate results can be reused instead of recomputed.
7. Intermediate data can be released when it is no longer needed.
8. The executed graph can be stored and reused for inspection or later execution.

This approach was chosen because it keeps the pipeline idea familiar to users who already know `scikit-learn`, but changes the internal execution model. Instead of treating the workflow as only a sequence of steps, `accelera_pipe` treats it as a graph. This helps the system handle more complex workflows that include multiple models, shared preprocessing, and branch comparisons.

3.3.3 Parallelizer

In this module, we decided to implement a conservative automatic parallelization approach. The goal is to speed up supported loop-heavy code by converting it to C++, analyzing its loops, inserting safe OpenMP pragmas, and compiling it into a native Python-callable extension.

The general framework of the approach is as follows:

1. Take the input source code, file, function, or transformer method.
2. Convert supported Python code into C++ when needed.
3. Extract loops using a compiler-based analysis stage.
4. Compute loop features such as memory access, dependency indicators, reduction patterns, branches, and function calls.
5. Classify whether the loop may be parallelized.
6. Apply rule-based safety checks before inserting any pragma.
7. Insert `#pragma omp parallel for` or a reduction pragma only when the loop is considered safe.
8. Compile the transformed C++ code into a native extension.
9. Return the optimized callable to be used from Python.

The chosen approach follows the same general idea as AI-assisted parallelization systems, but with a safer final pragma generation stage. The pragma generator is rule-based instead of fully AI-generated because the available data was limited and partly synthetic. This reduces the risk of generating unsafe OpenMP directives.

The Parallelizer is also connected to `accelera_pipe`. When a preprocessing node contains a supported custom Python function, the pipeline can attempt to optimize this function using the Parallelizer. This allows the project to combine graph-level workflow execution with code-level acceleration.

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

This section introduces the assumptions made for each module in Accelera and explains why these assumptions were needed during the implementation.

4.1.1 Accelera Pipeline Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.1.2 Auto Preprocessing Assumptions

The Auto Preprocessing module was designed based on the following assumptions:

- **Data assumption:** The input data should be valid, meaningful, and one of the supported types, so the module can select the correct preprocessing workflow.
- **Configuration assumption:** The user should provide the needed configuration values, so the module can apply the correct preprocessing steps.

4.1.3 Benchmark Assumptions

The Benchmark Platform was designed based on the following assumptions:

- **Benchmark data assumption:** The user is assumed to provide the benchmark files correctly, including the training data, test data, and separate target file. The user is also responsible for ensuring that the file separation does not cause data leakage, so the platform can evaluate submissions fairly.
- **Metric and submission assumption:** The admin is assumed to define the correct metric information for the benchmark, and the user is assumed to submit predictions in the required format, so the scoring script can calculate the result correctly.

4.2 System Architecture

4.2.1 Block Diagram

The block diagram in Figure 4.1 gives the big picture of Accelera. It shows the system as a group of connected modules rather than one single fixed pipeline. This is intentional: a user can use each module alone, or can connect several modules together to build an end-to-end machine-learning workflow.

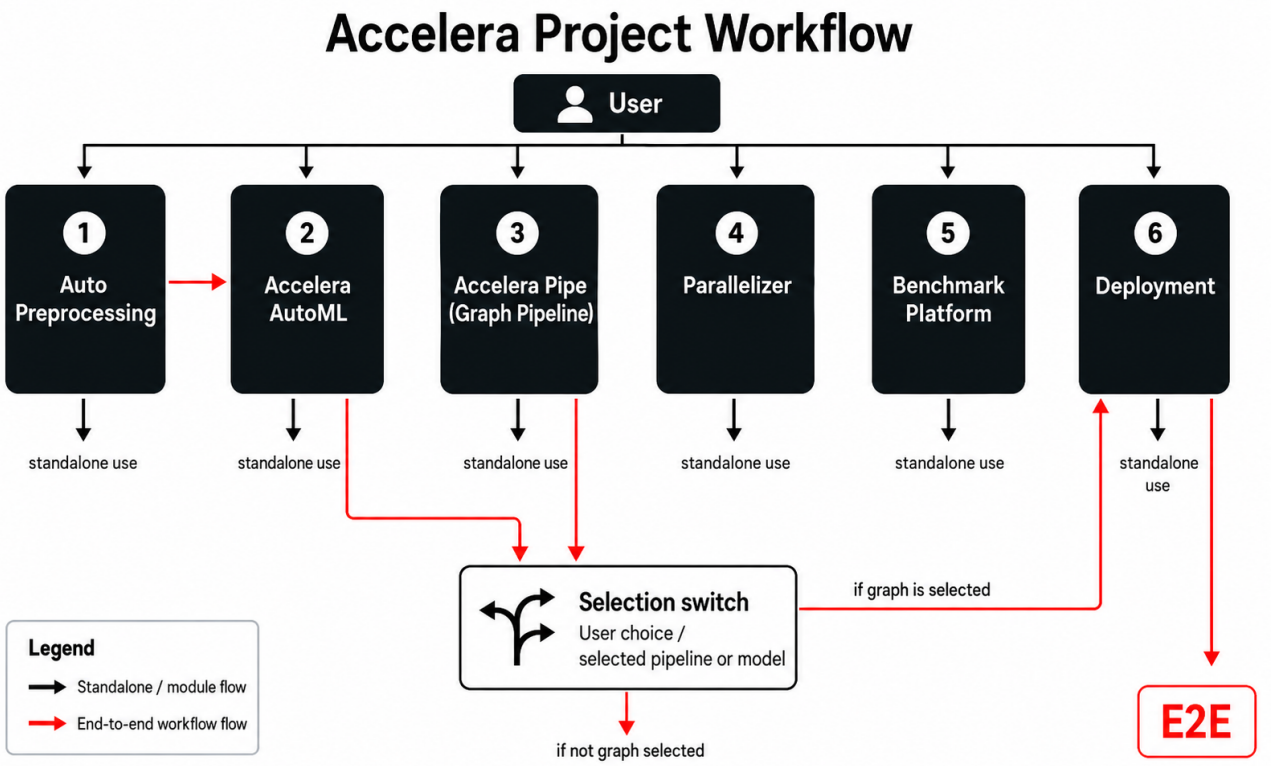

Figure 4.1: Overall block diagram of the Accelera project architecture.

At the highest level, the user can interact directly with the six main parts of the project: Auto Preprocessing, Accelera AutoML, Accelera Pipe, Parallelizer, Benchmark Platform, and Deployment. The blue arrows in the diagram represent standalone or module-level flows, while the red arrows represent the end-to-end workflow path.

Table 4.1: Main blocks in the Accelera architecture.

Block	Main responsibility	Output
Auto Preprocessing	Prepares datasets according to their type: tabular, text, or image.	Processed data, reports, and saved preprocessing artifacts.
Accelera AutoML	Searches for good model and preprocessing configurations.	Best model, score, leaderboard, and run history.
Accelera Pipe	Builds and executes graph-based machine-learning workflows.	Predictions, executed graph, selected path, and reports.
Parallelizer	Optimizes supported loop-heavy Python/C++ code using analysis, safety rules, OpenMP, compilation, and caching.	Parallelized C++ code or optimized Python-callable native module.
Benchmark Platform	Evaluates user prediction files against benchmark ground truth.	Score, ranking, and benchmark results.
Deployment	Turns a saved model or pipeline into a deployed service.	Model URL, service endpoint, and optional inference UI.

Auto Preprocessing flow

The Auto Preprocessing block starts from a dataset and tries to prepare it for machine learning usage. The module first profiles the data and detects its type. Then it applies preprocessing steps depending on the type of input.

- **Tabular data:** missing value imputation, categorical encoding, robust scaling, and feature selection.
- **Text data:** cleaning, tokenization, stemming, and TF-IDF feature extraction.
- **Image data:** image preparation for classification tasks.

The output can be used in two ways. In the standalone path, the user can directly use the processed data, reports, and saved preprocessors. In the end-to-end path, the processed data can be passed to the AutoML block for model search.

AutoML and selection flow

The AutoML block receives a dataset, extracts statistics and metafeatures, builds candidate models and preprocessing choices, and runs a search process. Its goal is to return the best model according to the selected metric.

The important point is that AutoML is not forced to be the only way to build a model. The user may choose an AutoML result, or may choose a graph-pipeline result from Accelera Pipe. This is why the diagram contains a selection switch.

Table 4.2: Possible paths to the deployment selection switch.

Path	Meaning
AutoML path	The user lets `accelera_automl` search for the best model, then chooses the selected model for deployment.
Graph pipeline path	The user builds a workflow using `accelera_pipe`, executes the graph, and selects the executed graph or best pipeline path for deployment.
Standalone path	The user may stop after any module and use its output directly without continuing to deployment.

Accelera Pipe and Parallelizer interaction

Accelera Pipe is the graph-execution part of the system. It represents the workflow as a graph of nodes. These nodes may represent preprocessing, model training, prediction, metric evaluation, merging, branching, and input/output operations.

A key feature is the interaction between Accelera Pipe and the Parallelizer. If a graph preprocessing node contains a supported custom method, the graph can send that method to the Parallelizer. The Parallelizer then tries to optimize it and return an optimized callable. After that, the optimized method can be attached back to the graph.

1. The user defines a graph workflow in `accelera_pipe`.
2. The graph detects a supported custom preprocessing method.
3. The method is sent to the Parallelizer.
4. The Parallelizer analyzes, rewrites, compiles, and caches the optimized implementation when possible.
5. The optimized method is returned and reattached to the graph.
6. The graph continues execution using either the optimized method or the original method if optimization fails.

This interaction is important because it connects workflow-level optimization with code-level optimization. Accelera Pipe manages the execution graph, while the Parallelizer attempts to speed up supported loop-heavy custom code inside that graph.

Parallelizer flow

The Parallelizer block accepts several kinds of input. This makes it usable both as a standalone module and as an internal acceleration module for Accelera Pipe.

- C/C++ file or source code.

- Python file or Python source code.
- Python function.
- Custom transformer method.

After receiving the input, the Parallelizer normalizes it. If the input is supported Python code, it is converted to C++ first. Then the module extracts loops using a Clang AST based analysis stage. After loop extraction, it computes features such as memory access, dependencies, reductions, branches, and function calls. These features are used with a classifier and rule-based safety checks to decide whether a loop can be parallelized.

Table 4.3: Parallelizer stages.

Stage	Description
Input normalization	Accepts source code, files, functions, or transformer methods and prepares them for analysis.
Python-to-C++ conversion	Converts supported Python code into C++ so it can enter the same loop-analysis path.
Loop extraction	Uses compiler-style analysis to locate loops and record loop ranges.
Feature extraction	Computes indicators such as reductions, dependencies, array access, branches, function calls, and vectorization potential.
Safety decision	Combines classifier output with deterministic rule-based checks.
Pragma insertion	Adds OpenMP pragmas only when the loop is considered safe.
Compilation and caching	Builds optimized native code when needed and reuses cached results when possible.

The final pragma generation is intentionally rule-based. This makes the behavior safer and easier to inspect, especially when the available data for fully AI-generated pragma generation is limited or synthetic.

Benchmark Platform flow

The Benchmark Platform is used for evaluation and ranking. It does not take a full user pipeline as input. Instead, it works with benchmark data links and user prediction submissions.

A benchmark owner creates a benchmark by providing the required CSV links:

- training CSV file link,
- testing CSV file link,
- true prediction or ground-truth CSV file link.

A participant submits:

- a predictions CSV file containing the prediction column,
- and optionally a GitHub repository URL for the project.

The platform compares the submitted prediction column with the true labels, computes the score, and updates the ranking. This makes the benchmark module focused on fair evaluation and leaderboard comparison, not on directly executing the user's pipeline.

Deployment flow

The Deployment block receives a saved model or pipeline. It prepares the deployment service, builds the required API layer, handles authentication or API configuration, publishes an endpoint, and optionally provides an inference UI.

The final deployment block produces a deployed model URL or service. This output has two possible meanings:

- **Standalone use:** the user deploys a saved model or pipeline directly and uses the endpoint.
- **End-to-end use:** the user selects a model or pipeline through the selection switch, deploys it, and receives the final E2E service output.

Overall architecture meaning

The main idea of the block diagram is that Accelera is modular but connected. Each module gives value by itself, but the full system becomes stronger when the modules are combined.

Table 4.4: Standalone use versus end-to-end use.

Mode	Description	Example
Standalone use	The user uses one module and stops at its output.	Use Auto Preprocessing only to generate processed data and reports.
End-to-end use	The user connects several modules together to move from data to deployed result.	Preprocess data, run AutoML or graph pipeline, select output, deploy service.
Integrated acceleration	A module internally uses another module to improve execution.	Accelera Pipe sends supported custom preprocessing methods to the Parallelizer.

Therefore, the block diagram communicates three important ideas. First, the system supports direct access to each module. Second, the modules can be connected

into a complete machine-learning workflow. Third, the interaction between Accelera Pipe and the Parallelizer allows the project to combine graph-based execution with source-level acceleration.

4.3 Accelera Pipe Module

4.3.1 Functional Description

The Accelera Pipe module can be described formally as a directed acyclic graph:

$$G = (V, E)$$

where V is the set of computational nodes and E is the set of directed dependency edges between them. Each node $v_i \in V$ represents one pipeline operation, such as preprocessing, model fitting, prediction, merging, or metric evaluation. Each edge $(v_i, v_j) \in E$ means that node v_j depends on the output of node v_i. To guarantee valid execution, the graph must satisfy the acyclic condition:

$$\nexists\, (v_1, v_2, \ldots, v_k) \quad \text{such that} \quad v_1 = v_k$$

This condition ensures that the graph contains no circular dependencies. Therefore, the backend can execute the pipeline using a valid topological ordering. If $\tau(G)$ denotes a topological order of the graph, then every dependency must appear before the node that consumes it:

$$u \to v \Rightarrow \tau(u) < \tau(v)$$

During execution, each node applies its stored operation to the outputs received from its parent nodes. For a node v_i, the output can be represented as:

$$o_i = f_i(o_{p_1}, o_{p_2}, \ldots, o_{p_m})$$

where f_i is the operation stored in the node, and $o_{p_1}, o_{p_2}, \ldots, o_{p_m}$ are the outputs of its parent nodes. This formulation allows both sequential and branched workflows to be handled by the same execution model.

For branch-based execution, the pipeline may produce several candidate paths. If $P = \{p_1, p_2, \ldots, p_n\}$ is the set of candidate paths and $M(p_i)$ is the metric value of path p_i, the selected path can be written as:

$$p^* = \arg\max_{p_i \in P} M(p_i)$$

for maximization metrics such as accuracy, or:

$$p^* = \arg\min_{p_i \in P} M(p_i)$$

for minimization metrics such as loss or error. The selected path p^* is stored inside the `ExecutedGraph`, which keeps only the fitted transformations and models needed for later inference.

At inference time, the executed graph applies only the selected fitted path to new input data. If X_{new} is the new input, the inference output can be represented as:

$$\hat{y} = G^*(X_{\text{new}})$$

where G^* is the fitted executed graph selected during training. This separation reduces unnecessary training-time objects and keeps inference focused on the final reusable path.

4.3.2 Modular Decomposition

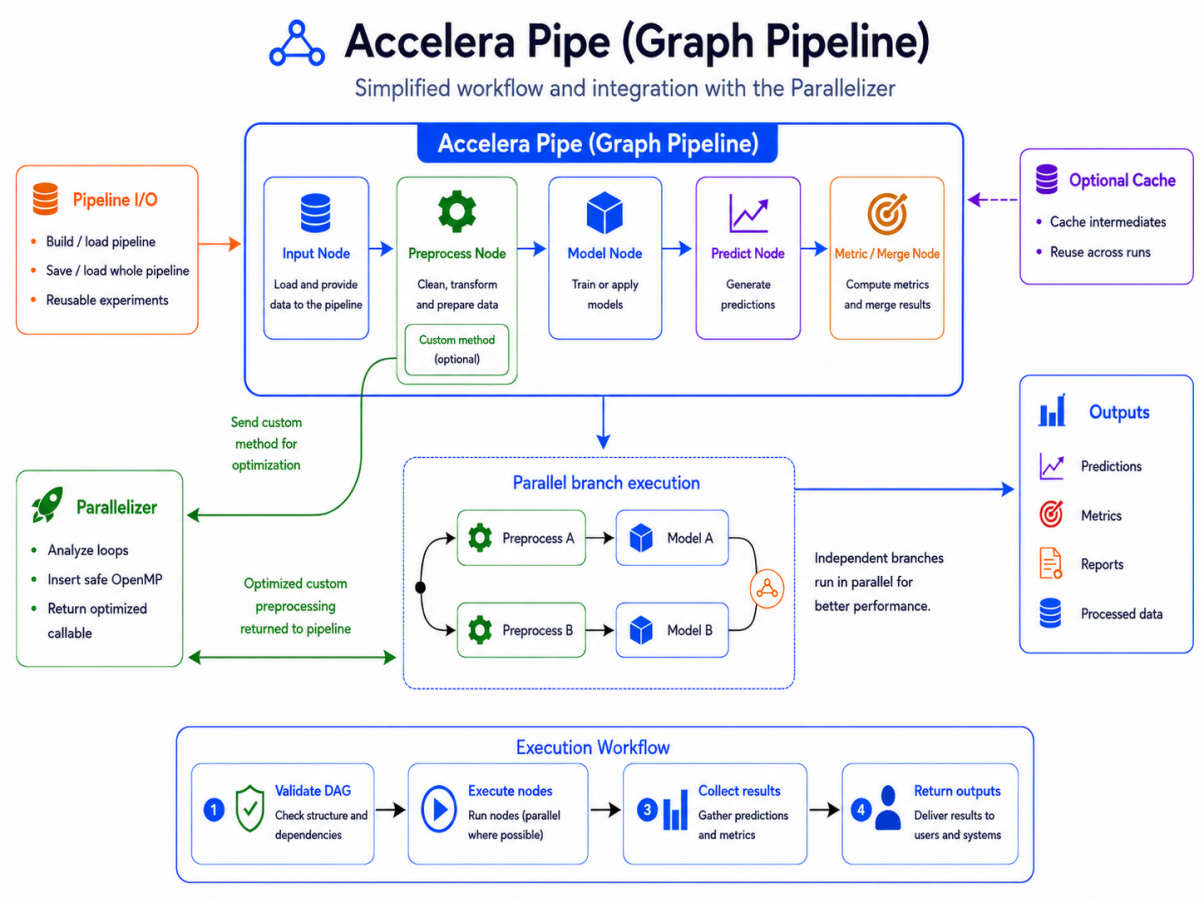

Figure 4.2: Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.

The Accelera Pipe module is decomposed into two main layers: a Python API layer and a native C++ graph execution layer. As shown in Figure 4.2, the Python layer provides the user-facing builder interface used to define preprocessing, model, prediction, metric, branch, and merge nodes. It also handles metric wrapping, reports,

serialization, save/load support, and integration with the Code Parallelizer.

The native C++ layer manages the internal graph representation and execution behavior. It stores nodes and edges, validates graph connections, computes the topological execution order, schedules independent branches for parallel execution, selects the final fitted path, and manages intermediate memory. When a preprocessing node contains a supported custom method, Accelera Pipe can send it to the Parallelizer, receive an optimized callable, and attach it back to the graph before execution.

Table 4.5: Main components of the Accelera Pipe module

Component	**Responsibility**
Pipeline Builder Interface	Provides the user-facing fluent API used to define preprocessing stages, models, prediction steps, metrics, merge operations, and branches. It translates each user call into a typed computational node instead of immediately executing the operation.
Pipeline State and Persistence Manager	Owns the native graph object, maps high-level node categories to graph node types, controls graph execution settings, and provides save/load behavior for reusable pipeline objects.
Executed Graph Runtime	Represents the fitted execution path selected after training. It allows the same trained transformations and models to be reused for inference without keeping unnecessary validation data or temporary training branches.
Metric and Display Wrappers	Adapt supervised metrics, unsupervised metrics, arrays, figures, dictionaries, and reports into graph-compatible outputs that can be stored, selected, displayed, or used in branch-selection strategies.
Native Graph Engine	Stores the pipeline as a directed acyclic graph, validates node connections, computes execution order, schedules sequential or parallel execution, manages branch selection, and controls intermediate-memory lifetime.
Node Abstraction Layer	Defines the common behavior of input, preprocessing, model, prediction, merge, and metric nodes, including source-node relationships, stored data, GPU usage flags, and selected-path markers.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Before a preprocessing function is stored, the module attempts to parallelize it through `parallelizer.parallelize`.

Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 4.6: Mapping from builder calls to graph node semantics

Builder method	**Node type**	**Main role**
`preprocess(name, func)`	`PREPROCESS`	Transform input data or selected columns.
`model(name, model)`	`MODEL`	Fit or apply a machine-learning estimator.
`predict(name, test_data)`	`PREDICT`	Run a model output function such as `predict`.
`metric(name, metric)`	`METRIC`	Evaluate predictions using a supervised or unsupervised metric.
`merge(name, strategy)`	`MERGE`	Combine outputs from multiple prediction branches.
`branch(name, ...)`	Multiple nodes	Create alternative candidate paths in the DAG.

4.3.3 Branching and Path Selection

The native backend uses a graph abstraction rather than a linear pipeline. The `Graph` class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running. This guarantees that each node is executed only after its source nodes have produced their outputs. If the graph contains branches, the execution result is followed by path selection using one of the supported strategies: maximum metric, minimum metric, custom strategy, or all paths.

Algorithm 4.1: High-level Accelera Pipe training flow

```
Input: training data X, optional labels y, graph G, selection strategy S
Output: predictions or metric results R, executed graph E

1:  Add X and y to the input node of G
2:  Compile G if its topological order is not already available
3:  Execute graph nodes sequentially or in parallel depending on G
settings
4:  Collect outputs from final leaf nodes
5:  If G contains branches, select the path requested by S
6:  Clone the selected fitted path into a compact executed graph E
```

```
7:  Remove validation-only data and temporary training data
8:  Return R and E
```

Branching is handled by the native `split` operation. Each branch is converted into a chain of graph nodes starting from the current leaf. If the current leaf node is L, then a branch B_i can be represented as:

$$B_i = (L, n_{i1}, n_{i2}, \ldots, n_{ik})$$

where $n_{i1}, n_{i2}, \ldots, n_{ik}$ are the operations added to that branch. A branch may contain a single node or a list of nodes, so the same mechanism supports simple model comparison as well as full alternative sub-pipelines.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects the later branch consumers to its output. The sharing condition is:

$$n_{11} = n_{21} = \cdots = n_{m1}$$

where n_{i1} is the first node of branch B_i. This is safe for the first node because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.

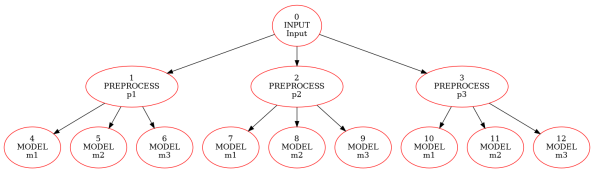

Figure 4.3: Before duplicate first-node sharing

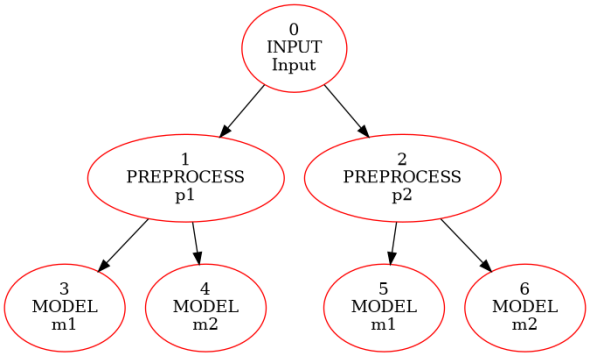

Figure 4.4: After duplicate first-node sharing

4.3.4 Intermediate Memory Management

Machine-learning pipelines can produce large intermediate arrays, especially when several preprocessing branches are evaluated. Accelera Pipe therefore tracks remaining consumers for each node output. Once the last downstream consumer finishes, the corresponding intermediate data can be released. Model and metric nodes are treated more carefully because they may hold fitted estimators or evaluation state, while input and preprocessing data can often be cleared after their consumers are done. This policy reduces memory pressure without changing the logical result of the graph.

Algorithm 4.2: Consumer-count based intermediate release

```
Input: executed node N, remaining consumer table C
Output: graph with unused temporary data released

1:  for each source node S that feeds N do
2:      C[S] = C[S] - 1
3:      if C[S] == 0 and S is releasable then
4:          clear the temporary data stored by S
5:      end if
6:  end for
```

The consumer-count mechanism is used after each node finishes executing. Every source node keeps a counter representing how many downstream nodes still need its output. For a source node S, the remaining-consumer count can be written as:

$$C(S) = |\{N \mid S \to N \text{ and } N \text{ has not executed yet}\}|$$

When a downstream consumer completes, the counter is decreased:

$$C(S) \leftarrow C(S) - 1$$

If the counter reaches zero and the source node is safe to clear, the temporary output is released. This allows the graph to reduce memory pressure during branch-heavy training runs without changing the final predictions or selected path.

4.3.5 Parallel Graph Execution

The graph can run independent nodes in parallel, but a node can start only after all its parents have finished. For any node N, the readiness condition is:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N),\ \text{finished}(P) = \text{true}$$

The backend uses CPU thread semaphores, a separate GPU semaphore, and dependency checks over the topological order. This prevents a downstream node from reading incomplete data and also serializes GPU work when necessary. CPU nodes are allowed to execute concurrently as long as worker slots are available, while GPU nodes are handled more conservatively using a single execution slot. This makes the scheduler safe for mixed CPU/GPU graphs without changing the dependency semantics. This also allows the graph to use available hardware more efficiently when several branches are ready at the same time. At the same time, synchronization keeps the final execution result equivalent to a valid sequential topological run. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERA_NUM_THREADS` environment variable.

Algorithm 4.3: Parallel graph execution scheduling

```
Input: topological execution order T
Output: completed graph execution or reported error

Initialize finished and started state for all nodes
Build remaining-consumer counts for intermediate outputs
Create limited CPU worker slots and one GPU execution slot
while unfinished nodes still exist do
    for each node N in T do
        if N has not started and all parent nodes are finished then
            Acquire a CPU slot or the GPU slot depending on N
            Mark N as started
            Launch N asynchronously
            After N finishes:
                Mark N as finished
                Release unused source outputs
                Release the CPU or GPU slot
         end if
     end for
end while
Wait for all launched tasks
Rethrow any stored node execution error
```

Algorithm 4.3 describes the runtime scheduling idea implemented by the native graph backend. The graph is already arranged in topological order, but the scheduler does not simply execute nodes one by one. Instead, it repeatedly searches for nodes whose parents have finished and launches them asynchronously when a CPU or GPU execution slot is available. CPU nodes share a limited pool of worker slots, while GPU nodes use a separate single-slot semaphore to avoid unsafe concurrent GPU execution. After a node completes, the backend marks it as finished, releases any temporary source outputs that are no longer needed, and signals the scheduler so that newly ready child nodes can start. Any exception raised inside a node is stored and rethrown after all launched tasks are joined, which gives a clear error message containing the failed node name and whether it was executed as a CPU or GPU node.

4.3.6 Design Constraints

The Accelera Pipe module has several design constraints that guide how the graph is constructed, executed, and reused. The first constraint is that the internal workflow must always remain a valid directed acyclic graph. Since execution depends on topological ordering, cyclic dependencies are not allowed. For any edge from node u to

node v, the execution order must satisfy:

$$u \to v \Rightarrow \tau(u) < \tau(v)$$

where τ represents the topological order of the graph. This guarantees that every node is executed only after its required source nodes have already produced their outputs.

The second constraint is node-type validity. Not every node can be connected to any other node. For example, a prediction node must follow a model node, a metric node must follow a prediction or merge node, and a merge node should combine compatible prediction outputs. These restrictions preserve the semantic meaning of the pipeline and prevent invalid workflows from reaching the execution stage.

The third constraint is safe memory management. Since branch-heavy workflows can produce many temporary arrays, the backend must release intermediate data only when it is no longer needed. For a source node S, the remaining-consumer count is:

$$C(S) = |\{N \mid S \to N \text{ and } N \text{ has not executed yet}\}|$$

The output of S can be cleared only when $C(S) = 0$ and the node is safe to release. This reduces memory pressure without changing the final selected path or prediction results.

The fourth constraint is parallel execution safety. Independent nodes may run in parallel, but a node can start only after all of its parent nodes have completed:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N),\ \text{finished}(P) = \text{true}$$

This prevents downstream nodes from reading incomplete data. The backend also uses limited CPU worker slots and a separate GPU slot to avoid unsafe resource sharing during execution.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned into an `ExecutedGraph`, validation inputs are removed, metric targets are cleared, and temporary arrays are released. This keeps the saved inference graph compact and focused only on fitted transformations and models.

4.3.7 Summary

The Accelera Pipe module also supports persistence. The Python `PipelineBase` class saves and loads pipeline objects using pickle, while the C++ graph exposes its state as a Python dictionary containing node information, selected-path flags, source indices, cached data, graph compilation state, and parallel execution configuration. During loading, the graph rebuilds the nodes and reconnects them using the stored source indices, making the native graph serializable.

The module can also export a graph-like representation of the pipeline structure. The graph utility code writes an XML-style network description containing layers for

input, preprocessing, model, prediction, merge, and metric nodes, together with their edges. This representation is useful for debugging, visualization, and understanding the executable graph generated from the high-level pipeline.

In summary, Accelera Pipe is a hybrid Python/C++ pipeline execution system. Python provides the user-facing construction interface, while C++ provides graph semantics, execution scheduling, branch selection, memory-lifetime control, and parallel execution support, keeping the module both efficient and easy to use.

4.4 Code Parallelizer Module

4.4.1 Functional Description

The Code Parallelizer module is responsible for transforming supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the scientific goal is the same: normalize the input into a C++ loop-analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

This process can be viewed as a transformation from an original program P into an optimized program P':

$$P' = \mathcal{T}(P)$$

where $\mathcal{T}$ represents the parallelization pipeline, including source normalization, loop extraction, loop classification, safety validation, and OpenMP pragma insertion. If no safe optimization is found, the transformation keeps the program behavior unchanged by returning the original code or callable.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same loop extraction path used for native C/C++ input. This design avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

After normalization, the module extracts candidate loops and checks whether each loop can be parallelized safely. For a loop L_i, the final decision can be represented as:

$$d_i \in \{\text{sequential},\ \text{parallel\_for},\ \text{reduction}\}$$

where sequential means that the loop is left unchanged, parallel_for means that a normal OpenMP parallel loop pragma can be inserted, and reduction means that the

loop can be parallelized using an OpenMP reduction clause. This decision is based on both learned classification and rule-based safety checks, so the module does not rely only on a model prediction when program correctness may be affected.

Table 4.7: Supported operations in the Python-to-C++ converter

Category	Supported forms and notes
Constants and names	`None`, booleans, integers, floats, strings, and variables.
Arithmetic	Addition, subtraction, multiplication, division, modulo, and power using `std::pow`.
Unary and boolean	Unary plus, unary minus, logical not, logical and, and logical or.
Comparisons	Equality, inequality, less-than, less-than-or-equal, greater-than, and greater-than-or-equal; chained comparisons rejected.
Calls and printing	Ordinary function calls and `print`; keyword arguments are generally unsupported.
Access operations	Attribute access and indexing are supported; slicing is rejected.
Assignment	Single-target assignment and indexed assignment are supported; multi-target assignment is rejected.
Augmented assignment	Simple-name `+=`, `-=`, `*=`, `/=`, and `%=`.
Control flow	`if`/`else` blocks and `return` statements.
Loops	`for i in range(...)` loops with one, two, or three range arguments.
Functions	Function definitions are emitted as C++ template-style functions; decorators are rejected.

After normalization, the module extracts loop information using native Clang AST bindings. Each loop is represented by its source code and source-line range. The Python orchestration layer then computes a fixed loop-feature representation, uses a classifier endpoint to predict whether the loop is a parallelization candidate, and validates the decision with rule-based checks. If the loop is safe, the module injects an OpenMP directive such as `#pragma omp parallel for`. For reduction-like loops, it can also emit a reduction clause when the reduction variable is valid.

The module has a fail-safe behavior. If the input cannot be converted, analyzed, classified, compiled, or loaded as a native extension, the system falls back to the original Python function or leaves the code unmodified. This is important because

parallelization is an optimization, not a reason to change the semantic meaning of the user's program.

4.4.2 Modular Decomposition

The Code Parallelizer module is decomposed into six cooperating components: input normalization, Python-to-C++ conversion, Clang-based loop extraction, feature extraction and vectorization, classification with rule-based validation, and OpenMP/native-extension generation.

Table 4.8: Main components of the Code Parallelizer module

Component	Responsibility
Parallelization Orchestrator	Receives source strings, file paths, custom functions, and transformer instances. It coordinates conversion, loop extraction, feature analysis, classification, pragma insertion, caching, compilation, and fallback behavior.
Input Normalization Layer	Converts accepted input forms into a common source-analysis representation. Files are read, source strings are analyzed directly, functions are captured from source, and transformer instances are inspected through their transformation method.
Python-to-C++ Converter	Translates a restricted subset of Python into C++ so Python preprocessing logic can enter the same static loop-analysis pipeline as native C/C++ code.
Loop Extraction Engine	Uses a compiler-style front end to locate loops, record source ranges, and expose structured loop information for feature extraction and rewriting.
Feature Extraction and Embedding Generator	Computes structural, dependency, memory-access, control-flow, arithmetic, and reduction features, then maps them into a fixed-size numeric representation.
Parallelization Classifier and Safety Resolver	Combines classifier predictions with deterministic safety rules to decide whether a loop remains sequential, receives a parallel-for pragma, or receives a reduction pragma.
OpenMP Rewriter	Inserts the selected OpenMP directive while avoiding unsafe duplicate insertion, especially when an outer loop already contains selected inner loops.
Native Compilation and Loading Layer	Wraps optimized C++ into a Python-callable extension, adds OpenMP flags only when needed, builds the native module, loads it at runtime, and returns the optimized callable.
Caching Layer	Stores processed source and compiled native functions using source-based hashes to avoid repeated conversion and compilation.

Figure 4.5 summarizes the overall data flow. In general, supported inputs are normalized into a common C++ loop-analysis path, while callable Python inputs additionally proceed to PyBind11 extension generation.

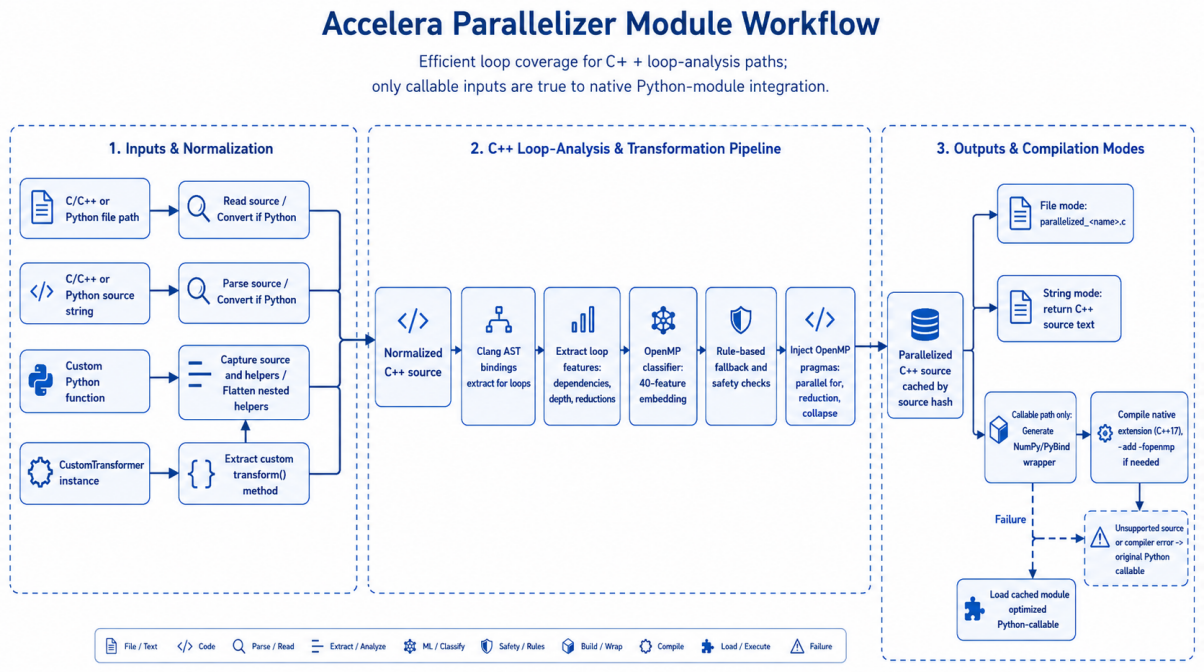

Figure 4.5: Code Parallelizer module workflow

The feature extraction stage analyzes both code structure and loop behavior, including reductions, array accesses, dependencies, indirect memory access, early exits, nesting depth, branches, function calls, arithmetic operations, and vectorization potential.

These indicators are encoded into a 40-dimensional vector for the classifier, then refined by deterministic safety rules that can still detect safe cases such as reductions, independent array writes, and nested loop patterns.

Algorithm 4.4: Loop classification and pragma insertion

```
Input: source program P
Output: source program P' with safe OpenMP pragmas

1:  if P is Python source then
2:      convert the supported Python subset into C++
3:  end if
4:  Extract for-loops using the Clang AST frontend
5:  for each extracted loop L do
6:      compute structural, memory, dependency, and reduction features
7:      send the 40-feature vector to the classifier service
8:      combine the predicted class with rule-based safety checks
9:      if the resolved class is parallel_for then
10:         insert #pragma omp parallel for
11:     else if the resolved class is reduction then
12:         insert #pragma omp parallel for reduction(...)
```

```
13:     else
14:         leave the loop unchanged
15:     end if
16: end for
17: Cache and return the transformed source code
```

The compilation path is used when the input is a Python callable. The parallelizer extracts or receives the function source, converts it to C++, parallelizes the C++ loop body, wraps the result in a PyBind11 module, compiles it as a shared native extension, loads the extension, and returns the optimized function. Caching is used at both the transformed-source level and the compiled method level. The cache key includes the compilation cache version, the function name, the compiler optimization level, the extension suffix, and the source code. This prevents recompiling the same function repeatedly.

4.4.3 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. Therefore, classifier output is not applied blindly. The module applies additional safety rules to avoid unsafe transformations when there are loop-carried dependencies, indirect accesses, early exits, side-effect function calls, or invalid reduction variables. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is not a weakness in the design; it is a correctness boundary. The system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11 headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. On non-Windows platforms it also uses position-independent code flags suitable for shared libraries. If compilation fails, the original Python callable is returned.

The fourth constraint is input and output compatibility. The current compiled callable path is oriented toward functions with one named input parameter and NumPy-like two-dimensional numeric data. The wrapper rewrites array access to PyBind11 unchecked NumPy access and exposes the compiled function back to Python. This makes the generated extension useful for numeric preprocessing loops,

but it also means that arbitrary Python functions with complex object structures are outside the supported optimization target.

The fifth constraint is reproducibility and caching. Because native compilation can be expensive, the module caches processed C++ code and compiled functions using source hashes. The cache must include enough information to avoid stale or incorrect reuse when compiler options, cache version, function name, source code, or extension suffix changes. This design improves repeated execution time while keeping cache invalidation explicit.

4.4.4 Summary

The Code Parallelizer is directly integrated with Accelera Pipe. When a preprocessing node is added, the pipeline calls the parallelizer before storing the preprocessing object. If the object is a custom Python function, it is wrapped as a source-backed function whose runtime callable can be replaced by the compiled implementation. If the object is a custom transformer instance, the parallelizer attempts to optimize its `transform` method and attach the optimized method back to the instance. If the object is a library transformer or an unsupported callable, it is returned unchanged.

Figure 4.6: Integration flow for compiled preprocessing inside Accelera Pipe

Scientifically, the final design uses a hybrid approach rather than a purely generative code model. An earlier generator-based design attempted to emit parallelized code using a neural sequence model. That approach was rejected because the dataset size was not sufficient for reliable sequence generation and because generated synthetic samples risked introducing distribution bias. The current design is more controlled: the classifier only predicts the type of parallelization opportunity, while deterministic compiler-like rules decide how and whether the pragma is inserted. This separation makes the system easier to inspect, easier to debug, and safer for user code.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely. This fits the broader Accelera design philosophy: keep the Python API simple, but move performance-sensitive and structurally analyzable work into optimized native execution when the system can do so without changing program semantics.

4.5 Auto Preprocessing Module

The Auto Preprocessing module automates the data understanding, exploratory data analysis, and preprocessing stages in the data science workflow. It helps reduce the repeated manual work needed to prepare datasets before modeling.

The module also helps the user understand the dataset before and after preprocessing by generating a report. This report shows important information about the input data and the preprocessing steps that were applied. For tabular data, the report shows information such as the number of rows and columns, data types, missing values, duplicate records, numerical and categorical features, basic statistics and EDA . For text data, it shows information such as the number of rows, target classes, class distribution, and number of missing. For image data, it shows information such as the number of images, image classes, class distribution, image sizes, and corrupted images. After preprocessing, the report shows the steps applied to the data, such as imputation, encoding, scaling, feature selection, text cleaning, TF-IDF feature extraction, image resizing, normalization, and image preparation for classification.

Another important design point is separating the training phase from the inference phase. In the training phase, the preprocessing objects are fitted on the training data and saved as artifacts. In the inference phase, these saved artifacts are loaded and applied to new data. This ensures that new data is processed in the same way as the training data before it is passed to the trained model for prediction.

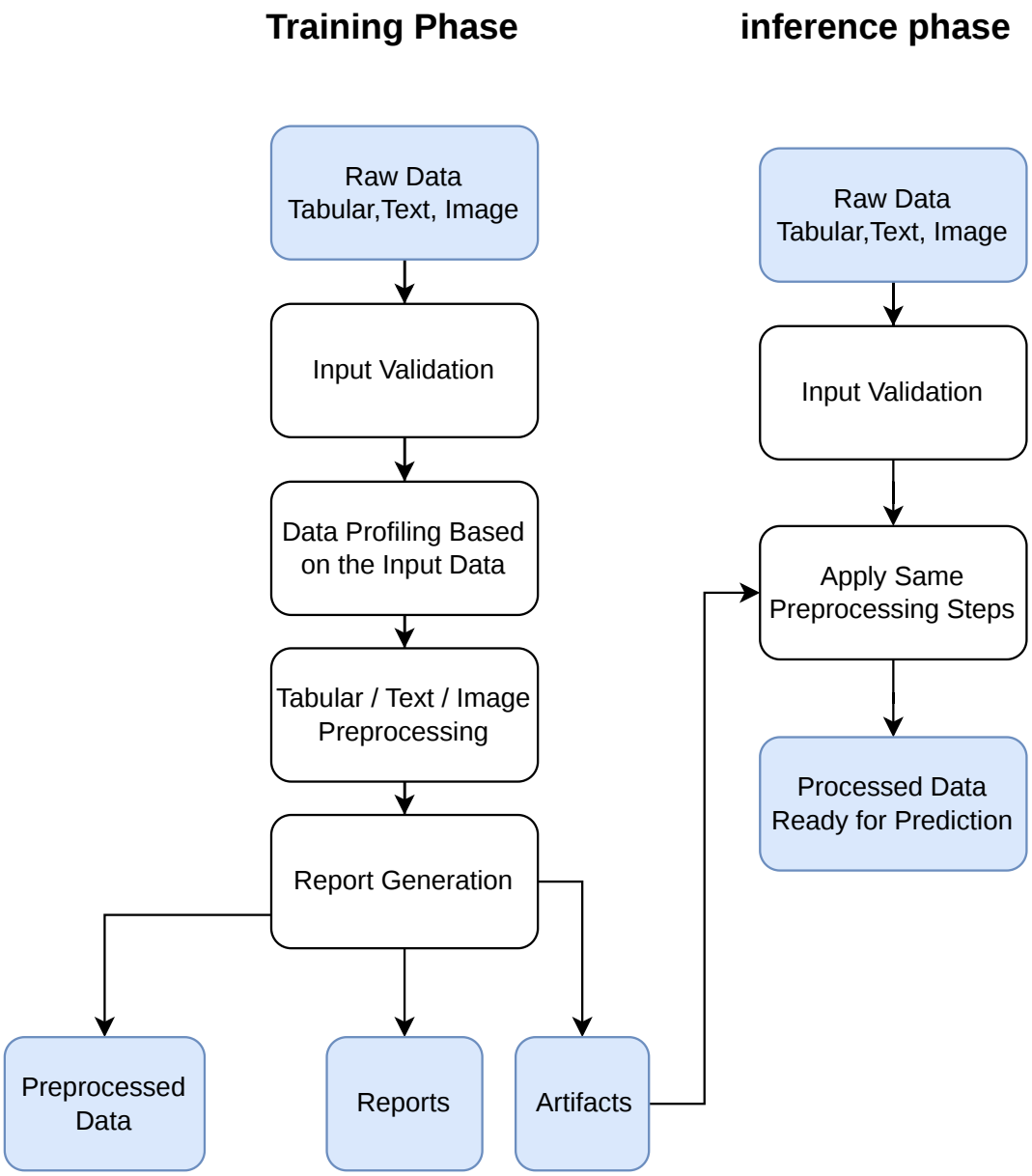

Figure 4.7: Auto Preprocessing Workflow

4.5.1 Functional Description

This module receives a dataset with its configuration, applies the suitable preprocessing workflow based on the data type and task, and returns processed data ready for training or inference. The module supports tabular, text, and image datasets. The module also generates a report when this option is enabled. The report summarizes information about the dataset and the preprocessing steps applied.

4.5.1.1 Tabular datasets

For tabular datasets it supports both classification and regression tasks. It analyzes the dataset, removes duplicate records, splits the data into training and validation sets, detects column types, handles missing values, encodes categorical features, scales numerical features, applies target preprocessing, performs feature selection, and saves the preprocessing objects needed later for test data.

4.5.1.2 Text datasets

It supports classification tasks. It cleans the text, tokenizes it, applies text preprocessing steps, and converts the text into a numerical representation suitable for machine learning models.

4.5.1.3 Image datasets

It supports image classification by reading images from class folders, validating image files, resizing images, applying augmentation, normalizing pixel values, and preparing the data loaders needed for training.

4.5.2 Modular Decomposition

The Auto Preprocessing module is divided based on the type of input data. Each data type has its own preprocessing workflow because every type has a different structure and needs different techniques:

- **Tabular data preprocessing:** handles structured datasets made of rows and columns.
- **Text data preprocessing:** handles text columns and converts cleaned text into numerical features.
- **Image data preprocessing:** handles image classification folders and prepares image loaders for training.

4.5.2.1 Tabular Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process.
- **handel_bool_types():** This function handles columns that have bool data type. It finds these columns and converts them into integer values.
- **data_overview():** This function gives a general overview of the dataset before preprocessing.
- **drop_target_nulls():** This function drops rows have null value in the target.

- **drop_duplicates():** This function removes repeated rows from the dataset.
- **split_data():** This function splits the dataset into training and validation sets. The split is based on the validation size and random state selected by the user. The output of this function is `X_train`, `X_val`, `y_train`, and `y_val`.
- **get_data_info():** This function studies each column in the training data. It collects important information such as data type, number of unique values, missing values, and missing percentage. For numerical columns, it also calculates the mean,and median.
- **is_drop_column():** This function decides whether a column should be removed or kept. A column is removed if it is included in the user selected drop list, if it has only one constant value, or if its missing value percentage is higher than the selected threshold.
- **drop_col():** This function removes the selected columns from both the training and validation datasets. It also saves the dropped columns and their reasons in a file. This helps the system apply the same dropping step later on testing or new data.
- **detect_column_types():** This function detects the type of each feature. This step is important because each feature type needs a different preprocessing method. The features are divided into binary columns, ordinal columns, numerical columns, continuous columns, low cardinality categorical columns, high cardinality categorical columns, and unsupported columns. The function also stores the preprocessing steps for each column to be shown later in the report.
- **make_graphs():** This function creates graphs when the report option is enabled. The type of graph depends on the feature type and the problem type.

Table 4.9: Detected Column Types in Tabular Data

Feature Type	Description	Examples
Binary	Columns containing exactly two unique values.	yes/no, 1/0
Ordinal	Integer columns with a small number of unique values less than or equal to `max_ordinal_unique`.	Ratings or levels such as 1, 2, 3, 4
Numerical	Integer columns with a number of unique values greater than `max_ordinal_unique`.	Age, number of transactions
Continuous	Floating point columns.	Height = 175.5, price = 19.99, temperature = 36.6
Low Cardinality Categorical	Object or string columns with a number of unique values less than or equal to `cardinality_threshold`.	Product categories, country group
High Cardinality Categorical	Object or string columns with a number of unique values greater than `cardinality_threshold`.	Product names, cities
Other	Columns with unsupported or non standard data types. These columns are excluded from the preprocessing pipeline.	Dates, complex objects, unsupported formats

Table 4.10: EDA graphs generated for tabular preprocessing

Column type	Problem type	Generated graphs
Binary and categorical	Classification	Null vs non null pie chart, category count plot, and category count split by target class.
Binary and categorical	Regression	Null vs non null pie chart, category count plot, and target box plot for each category.
Ordinal	Classification	Null vs non null pie chart, ordered count plot, and ordered count plot split by target class.
Ordinal	Regression	Null vs non null pie chart, ordered count plot, and target box plot for each ordinal value.
Numerical and continuous	Classification	Null vs non null pie chart, histogram with KDE, and feature box plot grouped by class.
Numerical and continuous	Regression	Null vs non null pie chart, histogram with KDE, and scatter plot between feature and target.
Target column	Classification	Null vs non null pie chart and target class count plot.
Target column	Regression	Null vs non null pie chart, target histogram with KDE, and target box plot.
Numeric columns	Both	Correlation heatmap.

- **features_preprocessing():** This function applies the preprocessing steps to the input features. It creates different pipelines based on the detected feature types:
  - Numerical and continuous features are filled using the median and scaled using robust scaling.
  - Binary features are filled using the most frequent value and encoded using ordinal encoding.
  - Low cardinality categorical features are filled using the most frequent value and encoded using binary encoding.
  - High cardinality categorical features are filled using the most frequent value and encoded using frequency encoding.
  - Ordinal features are filled using the most frequent value and scaled using robust scaling.

  These pipelines are combined using `ColumnTransformer`. The transformer is fitted on the training data and then applied to the validation data. The fitted preprocessor and the generated feature names are saved for later use.

- **target_preprocessing():** This function prepares the target column based on the problem type. For classification label encoding is used to convert class labels into numbers. For regression robust scaling is applied. The target preprocessor is saved so the same transformation can be used later.

- **features_importance():** This function selects the most useful features using mutual information with the target. Only features with scores greater than or equal to the selected threshold are kept. The selected features are saved in a file to use it in the inference step.

- **common_preprocessing():** This is the main function that controls the full tabular preprocessing workflow. It calls the previous functions in order. It starts by handling boolean columns, generating the data overview,drop rows have null target, removing duplicates, and splitting the data. Then, it extracts column information, drops unsuitable columns, detects column types, generates graphs, preprocesses the features, preprocesses the target, applies feature selection, and generates the report if enabled.It returns the processed training and validation data.

Algorithm 4.5: General tabular preprocessing flow

```
Input: DataFrame D, target column y, problem type, other configurations
Output: X_train, X_val, y_train, y_val

1:  Validate configuration
2:  Store original dataset columns
3:  Build data overview and report information
4:  Remove rows that have null target values
5:  Remove duplicate rows
6:  Split into training and validation sets
7:  Drop user-selected, high-missing, constant, and unsupported columns
8:  Detect column types from training data
9:  Generate feature and target graphs when report is enabled
10: Fit feature preprocessing objects on X_train
11: Transform X_train and X_val
12: Fit target preprocessing object on y_train
13: Transform y_train and y_val
14: Select useful features using mutual information
15: Save fitted objects and metadata
16: Generate report if enabled
```

Figure 4.8 shows the full tabular preprocessing workflow. It starts from the input dataset, applies data cleaning and feature preprocessing steps, then saves the processed training and validation data with the preprocessing objects needed later for testing.

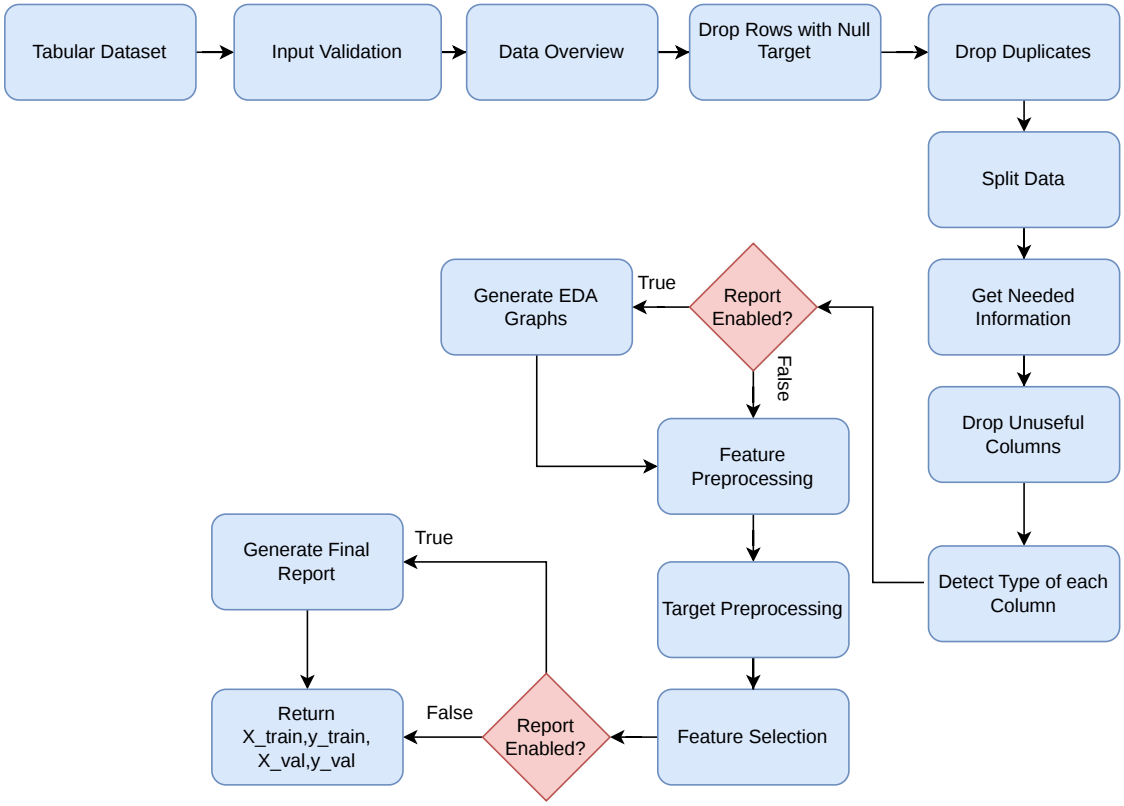

Figure 4.8: Tabular Preprocessing Workflow

4.5.2.2 Text Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process.Also removes any columns that are not the text column or the target column. This keeps only the columns needed for text classification. It also downloads the required NLTK resources, such as `punkt` and `stopwords`.
- **make_graphs():** This function creates graphs if the report option is enabled. It creates a graph for the text column . It also creates a graph for the target column. These graphs help the user understand the text data and the target class distribution.
- **custom_text_tokenizer():** This function is used to clean the text and split it into words. It converts the text to lowercase, removes extra spaces and unnecessary symbols, and handles some contractions such as `can't` and `won't`. It also separates numbers from letters. Then, the text is tokenized into words. Stopwords are removed, but words like `not` and `no` are kept because they affect the meaning. After that, stemming is used to reduce words to their basic form.
- **features_preprocessing():** This function applies the preprocessing steps to the text column. It fills missing text values in the training and validation data with

empty strings. Then it uses the custom tokenizer . It also stores examples of the text before and after cleaning in the report. Then `TfidfVectorizer` is used to convert the cleaned text into numerical features. The vectorizer uses the user settings, such as the maximum number of features, n-gram range, maximum document frequency, and minimum document frequency. The vectorizer is fitted on the training data and applied to the validation data. The fitted vectorizer is saved so the same text transformation can be reused later on testing or new data.

- **target_preprocessing():** This function prepares the target column for classification this function applies label encoding to the target values. The fitted label encoder is saved. Also, the text column and target column information are saved . This helps the system apply the same target transformation later on testing or new data.
- **common_preprocessing():** This is the main function that controls the full text preprocessing workflow. It calls the other functions in the correct order. It creates the data overview. Then, it removes rows that have missing target values and removes duplicate rows. Then, it splits the dataset into training and validation sets. If the report option is enabled, graphs are created. Then, feature preprocessing is applied to clean and vectorize the text. Target preprocessing is applied to encode the class labels. Generates the report if enabled.It returns the processed training and validation data.

Algorithm 4.6: Text tokenizer flow

```
Input: raw text sentence
Output: cleaned token list

1:  Convert text to lowercase and remove extra spaces
2:  Normalize apostrophe characters
3:  Expand common negative contractions
4:  Add spaces between letters and digits
5:  Remove characters except letters, digits, and apostrophes
6:  Split text into words
7:  Remove stopwords except "not" and "no"
8:  Remove very short tokens
9:  Apply Porter stemming
10: Return cleaned tokens
```

Figure 4.9 summarizes the text preprocessing workflow. It shows how the text data is cleaned, tokenized, and converted into TF-IDF features before the final processed data is returned.

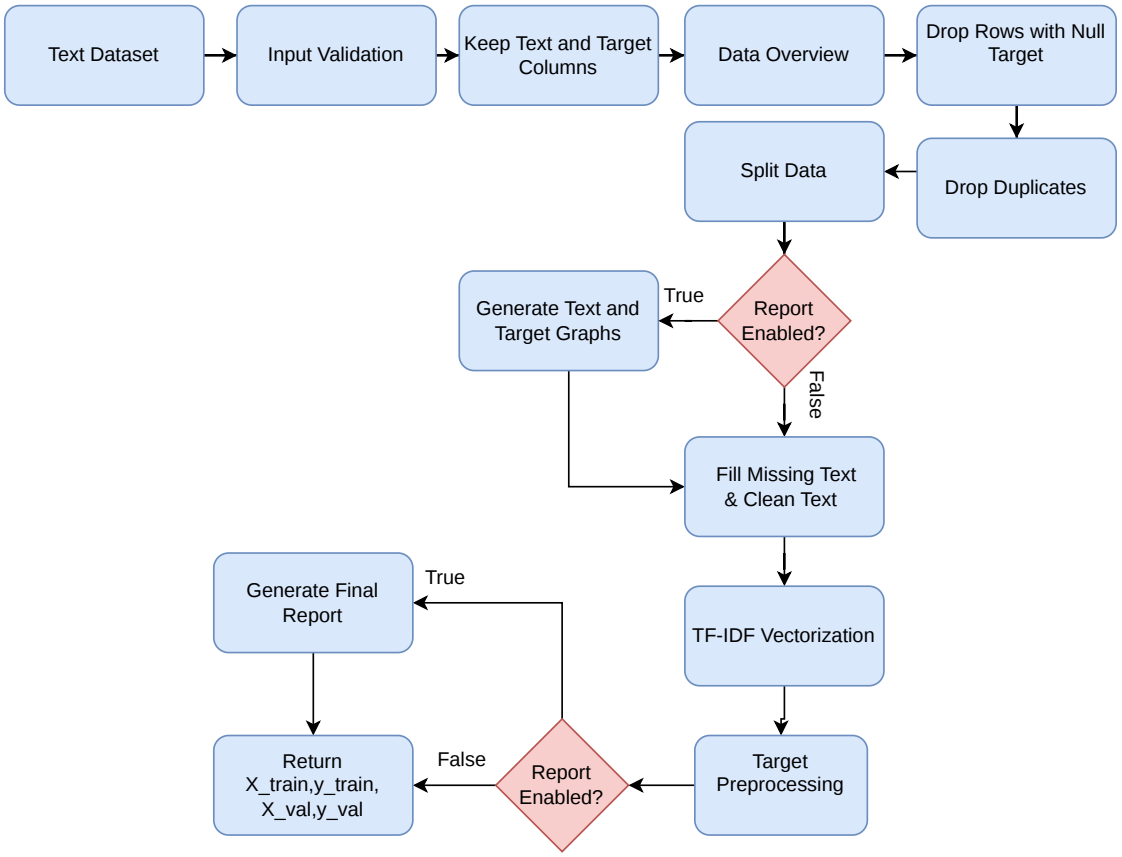

Figure 4.9: Text Preprocessing Workflow

4.5.2.3 Image Data Preprocessing

It is used for image classification datasets. In this module, the images are organized inside folders, where each folder represents one class. It is divided into a set of functions, and each function has a specific role in the image preprocessing workflow. The main functions are described as follows:

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process.It also prepares empty lists to store invalid images and their labels. It saves the image size information in a file so it can be reused later.
- **get_classes_mapping():** This function creates the class mappings. It gives each class name a numerical label.It creates two mappings: one from class name to label, and another from label to class name.These mappings are saved in files. This helps the system use the same labels during training, validation, testing, and prediction.
- **data_preparing():** This function reads the images from the class folders. It goes through each class folder and checks each image path.If the image is valid and

can be opened, its path is added to the list of valid image paths, and its class label is added to the labels list. If the image is invalid or corrupted, it is added to the invalid images list with its label. If there are no valid images, the function raises an error because the preprocessing process cannot continue without valid image data.

- **data_overview():** This function collects general information about the image dataset. For the training folder, it stores the class names, number of valid images, random sample images, number of invalid images, invalid image paths, and the class mapping. If a validation folder is provided, it also stores the same type of information for the validation folder. This information is used later in the final preprocessing report.

- **make_graphs_label_summary():** This function creates graphs that show the label distribution. It creates a label summary graph for the training folder. If a validation folder is provided, it also creates a label summary graph for the validation folder.

  If the training data is split into training and validation sets, the function also creates graphs for the labels after splitting. These graphs help the user understand if the classes are balanced or not.

- **make_garphs_sample():** This function creates sample image graphs. It shows random samples from the training folder. If a validation folder is provided, it also shows random samples from the validation folder.

  If the training data is split into training and validation sets, it also shows sample images after splitting. This helps the user visually check the image dataset.

- **make_graphs_loader():** This function creates sample images after using the DataLoader. It takes a small number of images from the training DataLoader and displays them.

  If a validation DataLoader exists, it also displays sample images from it. This step helps confirm that the images are loaded correctly after resizing, normalization, and augmentation.

- **make_graphs():** This function controls all graph generation steps. It prepares the graph section in the report, then calls the label summary graphs, sample image graphs, and DataLoader sample graphs.

- **get_loaders():** This function creates the training and validation datasets and DataLoaders. First, it creates a `ClassificationImageDataset` for the training images using the image paths, labels, image size, and augmentation settings.

  Then, it creates the training DataLoader with the selected batch size and uses shuffling. Shuffling is used for training because it helps the model see the data in a different order.

If validation data exists, the function creates a validation dataset and validation DataLoader. Augmentation is not applied to the validation data, and shuffling is not used because validation should be more stable.

- **common_preprocessing():** This is the main function that controls the full image preprocessing workflow. It starts by creating the class mappings. Then, it prepares the training images by collecting valid image paths and labels and saving invalid images.

  If a validation folder is provided, it also prepares the validation images. After that, it creates the data overview, splits the data if needed, creates the DataLoaders, generates graphs, and creates the final image preprocessing report.

  At the end, the function returns the training DataLoader and validation DataLoader. These outputs are ready to be used for training an image classification model.

Algorithm 4.7: Image training preprocessing flow

```
Input: training image folder, optional validation folder
Output: training and validation DataLoaders

1:  Read class folders from the training directory
2:  Create class-to-label and label-to-class mappings
3:  Save mappings and image size information
4:  Validate image files and skip invalid images
5:  Build the data overview for the report
6:  Split training images if no validation folder is provided
7:  Create training dataset with resize and optional augmentation
8:  Create validation dataset without augmentation
9:  Build DataLoaders
10: Generate label, sample, and loader graphs
11: Generate the image preprocessing report
```

Figure 4.10 explains the image preprocessing workflow. It shows how image folders are read, invalid images are detected, class labels are created, images are resized and normalized, and final DataLoaders are prepared.

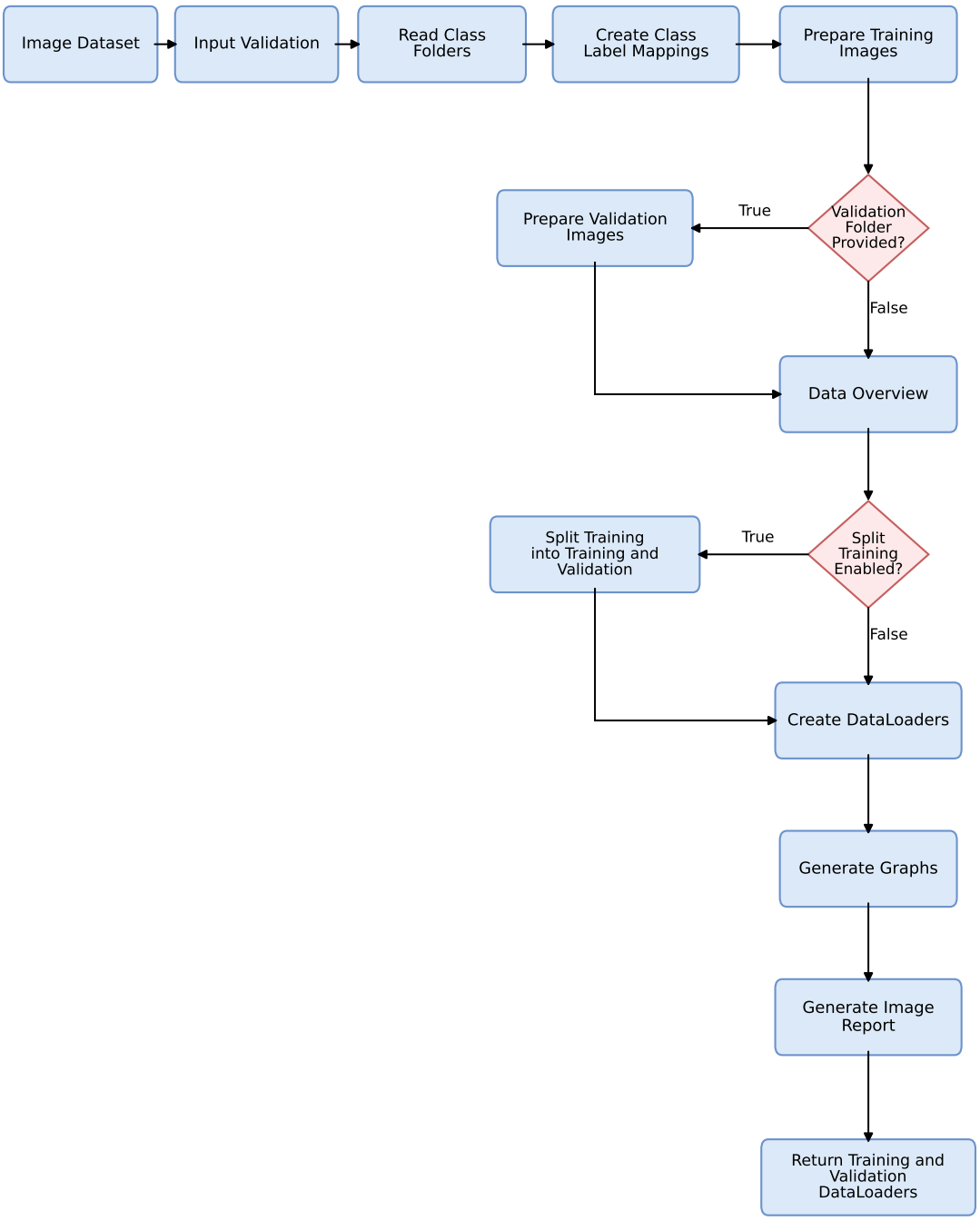

Figure 4.10: Image Preprocessing Workflow

4.5.2.4 Inference and Reusability:

For all supported data types, the saved preprocessing objects are reused during testing. This is important because the testing data must be processed in the same way as the training data.

- For tabular data, the system loads the saved preprocessors, selected features, dropped columns, and target preprocessor.
- For text data, the system loads the saved TF-IDF vectorizer and label encoder.
- For image data, the system loads the saved class mappings and image size.

This makes the preprocessing module reusable and consistent for training, validation, testing, and future prediction.

4.5.3 Design Constraints

The Auto Preprocessing module was designed under the following constraints:

- **Supported data types constraint:** The module is limited to the supported data types only, which are tabular, text, and image data. Other data types are not handled by this module.
- **Input structure constraint:** Each supported data type must follow the expected structure. Tabular and text data should be provided as structured datasets with a target column, while image data should be organized in folders based on class names.
- **Automatic decision constraint:** The module depends on rule-based preprocessing decisions. Therefore, the preprocessing choices are limited to the implemented rules and supported configuration options.

4.5.4 Module Summary

The Auto Preprocessing module provides one reusable preprocessing interface for tabular, text, and image data. It prepares data for training, generates reports when enabled, saves fitted preprocessing artifacts, and reuses the same artifacts during testing and prediction.

4.6 AutoML Module

4.6.1 Module Responsibility

The AutoML module automates model-family selection, hyperparameter search, preprocessing selection, and final model construction for supervised tabular classification

and regression. It exposes two Scikit-learn-style estimators: `AutoMLClassifier` and `AutoMLRegressor`. Both estimators support `fit`, `predict`, `score`, leaderboard inspection, configured scoring metrics, cross-validation folds, random seeds, time budgets, trial limits, meta-learning warm starts, and optional ensemble construction.

The main problem solved by this module is to produce a strong model under explicit time and trial constraints while avoiding unnecessary full-cost evaluations. The design separates public estimator behavior, task-specific evaluation, optimization, meta-learning, and ensembling so that each part can be extended independently.

4.6.2 Functional Description

The AutoML workflow starts by validating the input feature matrix X and target vector y. The estimator builds a task-specific engine, constructs a conditional configuration space, loads optional meta-learning warm starts, and evaluates candidate configurations through a staged optimizer. Each candidate contains a model family, active hyperparameters, and a preprocessing choice.

Every trial is evaluated using cross-validation and converted to an optimizer cost. For classification, accuracy is the default score and the cost is $1-$accuracy. For regression, R^2 is the default score and the cost is the negative score because the optimizer minimizes cost. Failed trials are recorded in the run history instead of terminating the complete search.

4.6.3 Configuration Space Construction

The search space is conditional. The active hyperparameters depend on the selected model family, so random forest parameters are active only when `random_forest` is selected, SVM parameters are active only for SVM models, and so on. The implementation stores model definitions in task-specific component registries. Each component defines its public name, hyperparameters, conditions, and estimator factory.

The engine can also filter models according to dataset properties. For example, some expensive models can be disabled for large datasets, and models that require non-negative features can be skipped when negative input values are detected.

4.6.4 Three-Stage Fidelity Schedule

To avoid spending full resources on weak candidates, the optimizer evaluates configurations through three fidelity stages:

$$f = (s, q, k, b),$$

where s is the stage index, q is the sample fraction, k is the number of cross-validation folds, and b is a multiplicative model budget. The current schedule uses:

- stage 0: 20% of samples, 2 folds, and 25% model budget;

- stage 1: 50% of samples, 3 folds, and 60% model budget;
- stage 2: full samples, configured folds, and full model budget.

Promising candidates are promoted to higher stages. This makes the search more resource-aware than evaluating every sampled configuration at full cost.

4.6.5 Surrogate-Guided Candidate Selection

After initial trials are collected, the optimizer vectorizes configurations and augments each vector with its fidelity variables. A random-forest regressor is trained as a surrogate model over observed costs. Predictions from individual trees provide an empirical mean and standard deviation. The optimizer ranks a sampled candidate pool using expected improvement:

$$EI(x) = (\mu^* - \mu(x) - \xi)\Phi(z) + \sigma(x)\phi(z), \qquad z = \frac{\mu^* - \mu(x) - \xi}{\sigma(x)},$$

where μ^* is the best observed cost, $\mu(x)$ and $\sigma(x)$ are the surrogate mean and uncertainty for candidate x, and Φ and ϕ are the normal cumulative distribution and density functions. This balances exploring uncertain regions with exploiting candidates predicted to improve the current best result.

4.6.6 Meta-Learning Warm Starts

The module computes basic metafeatures for the input dataset. Shared metafeatures include the number of instances and features, numeric and categorical feature ratios, missing-value ratios, skewness, kurtosis, and categorical symbol statistics. Classification adds class entropy and class probability statistics. Regression adds target mean, standard deviation, skewness, and kurtosis.

Packaged JSON metadata stores historical dataset descriptors and candidate configurations. At runtime, similar datasets are ranked by distance in the metafeature space, and the best historical configurations are converted into the current configuration space as warm-start candidates.

4.6.7 Parallel Execution and Timeout Control

The optimizer supports parallel candidate evaluation and per-trial timeout control. When timeout control is enabled, trials can be isolated in separate processes so that a slow or hanging estimator does not stop the parent search. The engine also passes internal job-count settings to estimators and ensembles to limit nested parallelism.

4.6.8 Final Model and Ensemble Selection

The strongest configuration is rebuilt and fitted on all available training data. If ensembles are disabled, this model is returned directly. If ensembles are enabled, successful

candidates from the run history are considered for voting or stacking.

For stacking, the module builds bagged base estimators and generates out-of-fold prediction blocks. Forward selection starts with the strongest single base model and adds candidates while they improve the meta-model score or until the minimum base-model requirement is satisfied. Classification uses a logistic-regression meta-model over probability blocks. Regression uses a ridge-regression meta-model over numeric prediction columns. The ensemble is added to the leaderboard and becomes the final model when its score is stronger than the best individual model.

4.6.9 Modular Decomposition

Table 4.11: AutoML implementation units

File or directory	Responsibility
`base.py`	Shared estimator lifecycle, validation, fitted state, leaderboard, and warning handling.
`classification.py`	Classification defaults and engine construction.
`regression.py`	Regression defaults and engine construction.
`core/automl.py`	Search orchestration, final model fitting, model filtering, and ensemble selection.
`components/`	Model registries, hyperparameter definitions, conditions, and estimator factories.
`configspace_search_space.py`	Task-specific configuration-space assembly and configuration conversion.
`evaluation.py`	Fidelity-aware estimator construction, preprocessing comparison, cross-validation, and cost conversion.
`optimization/smac_optimizer.py`	Trial scheduling, surrogate modelling, expected improvement, promotions, parallel workers, and timeouts.
`meta_learning/`	Metafeature extraction, similarity ranking, and warm-start retrieval.
`stacked_ensemble.py`	Classification adapters, bagging, forward selection, and stacked classification.
`stacked_ensemble_regression.py`	Regression stacking and forward selection.
`meta_learning_data/`	Packaged historical classification and regression metadata.

4.6.10 Design Constraints

- The current module targets supervised tabular classification and regression.
- Inputs must be compatible with Scikit-learn validation utilities.

- Optional libraries such as XGBoost, LightGBM, and CatBoost are used only when available in the environment.
- Trial failures and estimator exceptions must be recorded rather than crashing the full search.
- Search should remain reproducible when a random state is provided.
- Warm-start metadata must be stored relative to the package so it works after installation.

4.6.11 User-Facing Example

Algorithm 4.8: Using Accelera AutoML for classification

```
from accelera.src.accelera_automl import AutoMLClassifier

model = AutoMLClassifier(
    time_budget=300,
    n_trials=20,
    cv=3,
    random_state=42,
    use_meta_learning=True,
    use_ensemble=True,
    ensemble_strategy="stacked",
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(model.score(X_test, y_test))
print(model.return_leaderboard(top_n=3))
```

4.7 Deployment Module

4.7.1 Module Responsibility

The Deployment module is responsible for transitioning trained machine learning models from the local training environment to production-ready serving environments. It packages model binaries and preprocessing pipelines, validates incoming API request schemas at runtime, tracks prediction telemetry, provides local and remote deployment targets (Docker, Heroku, AWS EC2), and maintains a lightweight model version control system (VCS).

The core files under `accelera/src/deployment/` are:

- `server.py`: The FastAPI web server exposing standard and batch inference endpoints.

- `modelservice.py`: Manages model loading, preprocessor execution, and prediction routing.
- `schema_validation.py`: Performs runtime validation and data coercion using Great Expectations.
- `tracking.py`: Records asynchronous prediction inputs, outputs, and latencies.
- `deployment.py`: The orchestrator containing CLI commands for Local, Heroku, and EC2 deployments.
- `vcs.py`: The model version control system managing staging, commits, and rollbacks.

4.7.2 Modular Decomposition

4.7.2.1 Model Serving Subsystem

Model serving is built on FastAPI and Uvicorn to handle concurrent HTTP requests. The subsystem loads the model and preprocessors via `ModelService`, which inspects the configuration (`config.json`) to locate the serialized artifacts. During execution, it runs input data through any fitted preprocessors in sequence before invoking the model's prediction method. A web-based graphical user interface (GUI) is dynamically rendered based on the expected input features, allowing users to submit single predictions interactively. Figure 4.11 shows the block diagram of the model serving architecture.

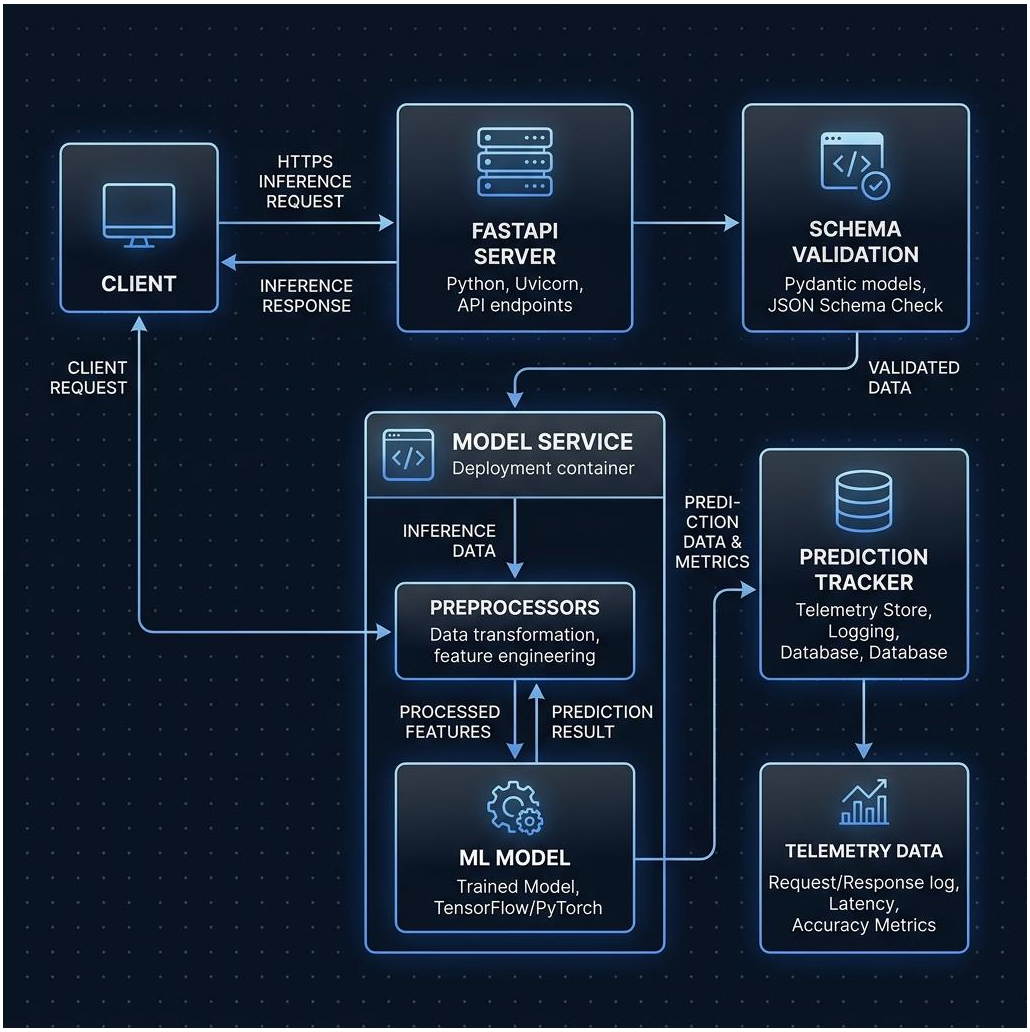

Figure 4.11: Model Serving Architecture and Execution Flow

The following methods and files implement the serving logic:

- `modelservice.py`:
  - `ModelService.load()`: Parses `config.json` to discover active model configuration. It opens and loads the main estimator file (e.g. `model.pkl`) and any auxiliary preprocessors (e.g. scalers or encoders) from the models directory.
  - `ModelService.predict(input_data, validate=True)`: Performs prediction. If validation is active, it calls `validate_input(input_data)` to receive validated and coerced values, converts the inputs into a NumPy array, adjusts dimensions, applies all loaded preprocessor transforms sequentially, and calls the underlying estimator's `predict()` method.
- `server.py`:
  - `_predict(input_data, endpoint, filename=None)`: Coordinates serving telemetry. It records starting timestamps using `time.perf_counter()`, exe-

cutes model prediction via `ModelService.predict()`, calculates API response latencies, and logs metrics to the telemetry storage.

- `/predict` and `/predict/csv` Endpoints: Standard HTTP POST methods. The first endpoint receives a structured JSON payload, while the second parses uploaded CSV files into Pandas DataFrames.
- `/gui` and `/health` Endpoints: The GUI endpoint returns the dynamic interactive testing form via `_render_gui(schema)`, while the health endpoint serves load balancers with standard status checks.

4.7.2.2 Runtime Schema Validation

To prevent invalid inputs from causing model crashes or silent degradation, incoming payloads are validated against a schema specified in the configuration. The validation layer uses Great Expectations to construct an ephemeral data source context. Columns are validated for expected names, missing entries, unexpected inputs, and value bounds (e.g., minimum and maximum thresholds). Data types are coerced automatically (e.g., matching string, boolean, integer, or floating-point formats) to match the training format, raising a structured `SchemaValidationError` if coercion fails.
The schema validation system is implemented in `schema_validation.py`:

- `InputSchema.__init__(config)`: Extracts features and limits from configuration. If schema validation is enabled, it requests a Great Expectations context (`gx.get_context()`), generates a unique UUID-based data source, registers a pandas-based DataFrame data asset, and configures a whole-dataframe batch definition.
- `InputSchema.validate(input_data)`: Normalizes the input payload into a DataFrame. It checks for expected column presence, detects extra or missing features, runs the Great Expectations expectation suite, and executes type coercion.
- `InputSchema.coerce_columns(df, errors)`: Loops through features to coerce types. It casts strings to numerical columns using `pd.to_numeric(..., errors='coerce')`, checks for integer constraints (`value % 1 == 0`), and processes boolean symbols (such as replacing `"true"`/`"false"` with python booleans).
- `InputSchema.to_dataframe(input_data)`: Utility method to convert list, dict, or DataFrame structures into standard Pandas format.

4.7.2.3 Telemetry and Request Tracking

Production deployments capture inference requests and runtime metrics using a lightweight tracking layer. When enabled, the tracker logs prediction inputs, model outputs, processing latencies (measured in milliseconds), and timestamps. To avoid

performance degradation on the prediction path, logging is written incrementally to a JSON lines (`.jsonl`) file.
Telemetry tracking is implemented in `tracking.py`:

- `PredictionTracker.__init__(config)`: Sets up the file system paths and creates/opens the telemetry log file.
- `PredictionTracker.log_prediction(inputs, outputs, latency, status)`: Formats request parameters, response targets, server latency, and datetime strings into a single-line JSON string and appends it to the logging stream.

4.7.2.4 Deployment Pipeline Orchestration

The orchestration engine automates containerizing and launching the service across three distinct environments. It stages the codebase, copies project dependency requirements, and builds dockerized containers to ensure portability and reproducibility across platforms.

1. **Local Docker**: Generates a clean `Dockerfile` using a Python slim base image, stops any conflicting containers on the target port, builds the container image locally, and runs it.
2. **Heroku**: Authenticates with the Heroku CLI and pushes/releases the Docker container using Heroku's Container Registry.
3. **AWS EC2**: Creates an SSH transport connection and synchronizes the staging directory to the target host using `rsync` (excluding Python cache directories). It installs and starts the Docker daemon, builds the container on the instance, and executes local health check probes before verifying the public web endpoint.

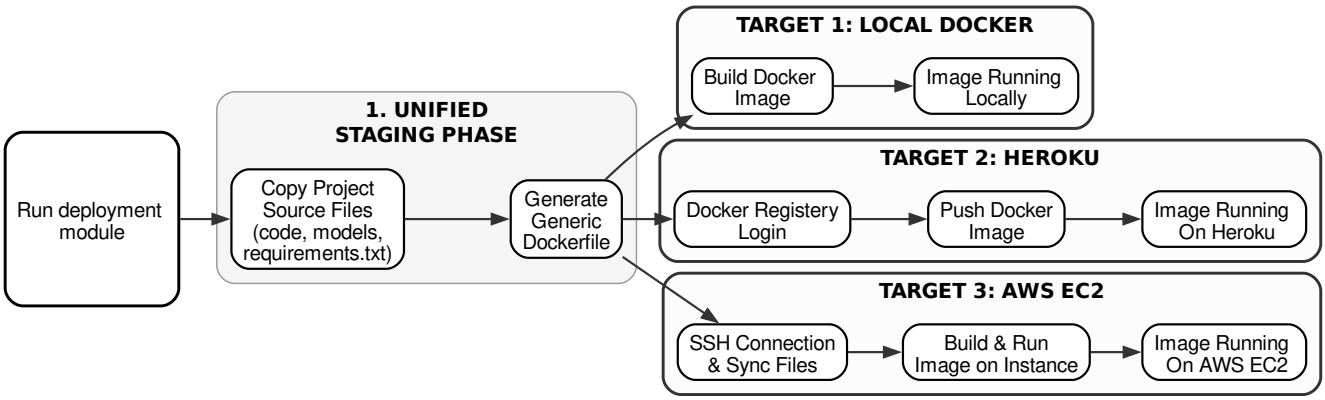

Figure 4.12: Multi-Target Deployment Orchestration Flow

4.7.2.5 Model Version Control (VCS)

The internal VCS provides reproducibility for deployed pipelines. It calculates a stable SHA-1 hash for each commit based on its timestamp and commit message. Models and configurations are archived into an `experiments/` subdirectory. The system maintains a global index (`experiments.json`) containing parent-child commit histories, HEAD references, and active deployed markers, facilitating rollbacks and deployments of historical model versions.
The version control subsystem is implemented in `vcs.py`:

- `calculate_hash(timestamp, message)`: Computes the 7-character SHA-1 hash value that acts as the commit identifier.
- `init(args)`: Configures the repository workspace by creating directories and saving the default index (`experiments.json`). `commit(args)`: Validates that models exist, generates a commit hash, copies the active configuration and models to `experiments/<hash>/`, and appends the commit entry to the registry.
- `log(args)`: Lists all recorded commits, showing details such as commit hash, creation date, message, and deploy status.
- `show(args)`: Displays the details and settings of a target commit.
- `deploy(args)`: Deploys a specific commit by copying its stored model files and configuration back into the active runtime workspace.

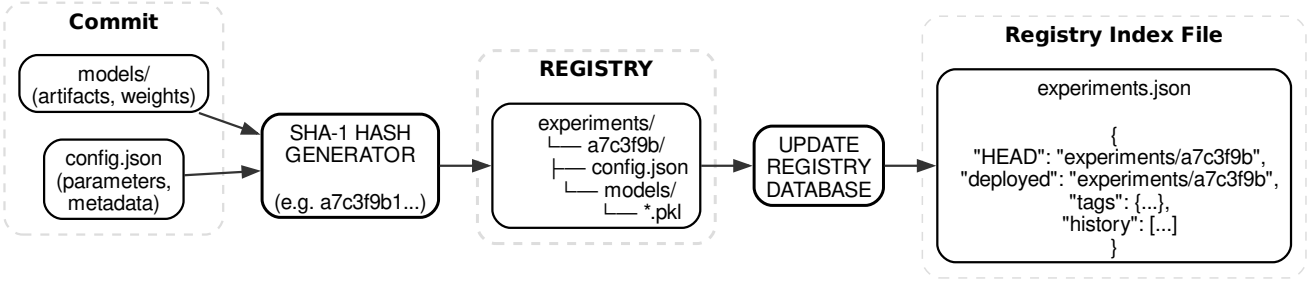

Figure 4.13: VCS Commit Workflow

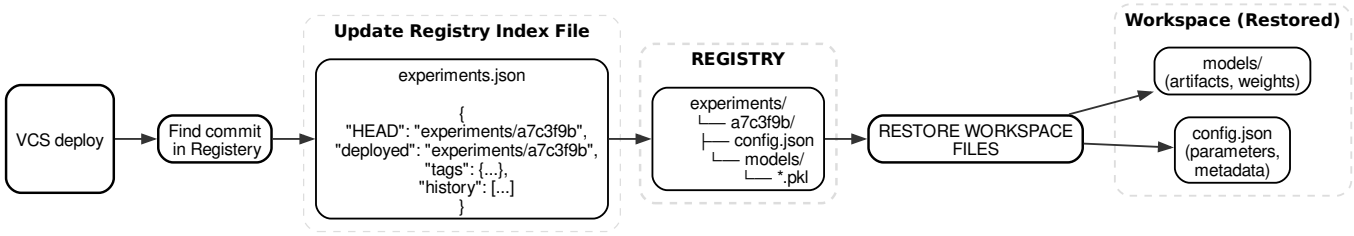

Figure 4.14: VCS Deploy Workflow

4.7.3 Command-Line Interface and CLI Commands

The orchestrator and version control features are exposed to the user through command-line interfaces inside `deployment.py` and `vcs.py`. The supported commands and their behaviors are:

4.7.3.1 Orchestrator Commands

- **prepare**: Prepares the deployment staging area. It copies service modules (e.g. `server.py`, `modelservice.py`) and the `accelera` source library to the `accelera_deployment` folder, and generates the serving `Dockerfile` and `requirements.txt`.
- **build**: Builds the local Docker container. It invokes local subprocess shell commands (e.g. `docker build -t <image_name> .`) with an optional `-no-cache` flag.
- **run-local**: Starts the local Docker container. It stops and removes any conflicting containers on the port before launching the container using port mappings.
- **local**: Orchestrates the complete local container pipeline by sequentially running the **prepare**, **build**, and **run-local** steps.
- **heroku-login**: Logs the client environment into the Heroku CLI and Heroku Docker registry.
- **heroku-deploy**: Packages and deploys the container to Heroku. It triggers login, pushes the docker image to Heroku's registry, and releases the container for traffic.
- **heroku-push** and **heroku-release**: Granular command calls that push the container image and release it separately.
- **ec2-deploy**: Deploys to a remote AWS EC2 instance. It establishes an SSH connection, synchronizes files using `rsync`, compiles/builds the Docker container on the host, runs it, and probes local health targets.
- **ec2-status**, **ec2-logs**, and **ec2-stop**: Operational tools to verify container execution, read Uvicorn outputs, and terminate the container on the target EC2 node.

4.7.3.2 Version Control Commands

- **init**: Sets up a new model version control workspace by initializing directories and registries.
- **commit**: Snapshots active model parameters and configurations under a message flag (`-m`).
- **log**: Prints the history log of all model commits.
- **show**: Prints the configuration dictionary details of a target commit.
- **deploy**: Restores a specified commit back into the staging area for subsequent deployment execution.

4.7.4 Design Constraints

The design of the Deployment module is subject to several key constraints:

- **Package Size Optimization**: To minimize container build times and storage footprints, heavy libraries like PyTorch or TensorFlow are excluded from the serving runtime unless explicitly required by the model.
- **Non-Interactive Execution**: Staging and EC2 orchestration must run headlessly in CI/CD pipelines, necessitating automatic Docker installs and automated SSH host key acceptance (`StrictHostKeyChecking=accept-new`).
- **I/O Isolation**: Process and SSH connection errors are caught as base `OSError` types to provide descriptive feedback on firewall and network issues without interrupting local execution.

4.8 Benchmark Platform Module

The Benchmark Platform module is designed to provide a shared environment where users can create machine learning benchmarks, submit predicted results, and compare submissions using selected evaluation metrics. The module includes a backend API, database schemas, Python validation and scoring scripts, and a frontend interface for users and admins.

The module separates benchmark creation from submission evaluation. This makes the workflow clearer: first, the benchmark creator defines the task and evaluation rules; then, users submit predictions under the same conditions. This separation helps ensure that all users are evaluated using the same ground-truth file and the same metric configuration. Another important design point is the use of Python scripts for file validation and metric scoring. The backend is implemented using Node.js and Express, while Python handles the machine learning evaluation part through Scikit-learn metrics and dataset loading utilities.

4.8.1 Functional Description

The main function of this module is to support fair comparison between different machine learning solutions. A user can create a benchmark by providing the benchmark title, description, problem type, target column, dataset link, test dataset link, ground-truth target link, and evaluation metric. The system validates the provided links and checks that the test set does not contain the target column while the target file contains the expected target column.
After a benchmark is created, other users can submit their prediction file and repository link. The system validates the submitted prediction file, calculates the score using the selected Scikit-learn metric, stores the submission, and displays submissions on a leaderboard. The leaderboard is ordered based on the metric configuration, where some metrics are better when their value is higher and others are better when their value is lower.

4.8.2 Modular Decomposition

The Benchmark Platform is decomposed into several smaller components:

- **Benchmark Management:** This component handles benchmark creation, listing, filtering, viewing, and deletion. It stores benchmark information such as title, description, problem type, dataset links, target column, selected metric, metric parameters, creation date, and creator.
- **Metric Management:** This component allows admin users to define evaluation metrics. Each metric contains a display name, the corresponding Scikit-learn metric function name, needed parameters, problem type, and whether a higher or lower score is better. Metrics are linked to benchmarks and used during submission scoring.
- **Submission Management:** This component allows users to submit prediction files for a specific benchmark. Each submission stores the benchmark id, submitting user, repository link, prediction file link, score, and submission date. Users can also update their existing prediction file or delete their submission if they have permission.
- **Validation and Scoring Scripts:** The backend calls Python scripts to validate benchmark files and score submissions. During benchmark creation, the validation script checks that the test file does not include the target column and that the target file contains the required target column. During submission, the scoring script loads the ground-truth file and the submitted prediction file, checks their length and target column, then computes the selected Scikit-learn metric.
- **Leaderboard Interface:** The frontend displays benchmark submissions in a leaderboard. It shows the submitter information, repository link, score, and sub-

mission date. The sorting direction depends on the selected metric, so the platform can support both metrics where higher is better and metrics where lower is better.

Algorithm 4.9: Benchmark creation flow

```
Input: benchmark form data
Output: saved benchmark or validation error

1:  Receive benchmark creation request
2:  Validate problem type
3:  Check title uniqueness
4:  Validate dataset, test set, and target label links
5:  Run validate_user_links.py
6:  If the test file contains the target column, reject benchmark
7:  If the target file does not contain the target column, reject
benchmark
8:  Save benchmark document in MongoDB
```

Algorithm 4.10: Submission scoring flow

```
Input: benchmark target file, submitted prediction file, metric name
Output: score or validation error

1:  Retrieve true target file
2:  Retrieve submitted prediction file
3:  Check that both files have the same number of rows
4:  Check that submitted file contains the target column
5:  Load metric function from sklearn.metrics
6:  Compute metric(true_target, predicted_target, metric_parameters)
7:  Return score to the backend as JSON
8:  Save submission with score
```

4.8.3 Design Constraints

Several constraints affected the design of the Benchmark Platform:

- **Task and data constraint:** The platform is limited to classification and regression benchmarks that use tabular files. This includes the training data, test data, prediction file, and target file.
- **Storage and access constraint:** Dataset and prediction files are handled through accessible links to reduce server storage usage.

- **Scoring constraint:** The platform uses Scikit-learn metrics for scoring. Therefore, the selected metric must be supported by Scikit-learn and must match the benchmark problem type.

4.8.4 Module Summary

The Benchmark Platform module provides a controlled environment for benchmark creation, prediction submission, validation, scoring, and leaderboard ranking. It keeps the web management logic in the backend and frontend, while the metric calculation logic is handled by Python scoring scripts.

Chapter 5: System Testing and Verification

This chapter explains how the project was tested to make sure that the main parts work correctly. The testing was not done in one way only, because each part of the project has a different output. The main purpose of testing was to make sure that each part can receive the input, apply the required steps, save the needed files, and return the expected output without errors.

5.1 Testing Setup

The testing was done mainly in a Python environment. The main tools used in testing were Python, Pytest, Pandas, NumPy, Scikit-learn, PyTorch, Matplotlib, and the libraries used in the implementation. For the benchmark platform, Node.js, Express, React, MongoDB, and Python scoring scripts were also checked. Some tests were written and run automatically using Pytest. Other parts were checked manually by comparing the actual output with the expected output.

5.2 Testing Plan and Strategy

The testing plan was based on testing the project step by step. First, each module was tested separately. After that, the outputs were checked in a more complete workflow. For example, preprocessing output was passed to machine learning models, generated reports were opened and inspected, and benchmark submissions were checked after scoring.
Different testing methods were used depending on the part being tested. Some parts were tested using unit tests with Pytest. Other parts were checked manually by comparing the actual output with the expected output. In some cases, the results were compared with another tool, to check if the preprocessing results were reasonable.

5.2.1 Module Testing

5.3 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.1: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.1 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.2: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.2 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.3: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler → SVC	0.7900	0.8250
5,000	MinMaxScaler → SVC	0.9160	0.9100
10,000	MinMaxScaler → SVC	0.9385	0.9260
50,000	MinMaxScaler → SVC	0.9565	0.9598
100,000	StandardScaler → SVC	0.9675	0.9709
250,000	StandardScaler → SVC	0.9774	0.9768

Table 5.3 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.4: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.4 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.5: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.5 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.6: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.6 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×.

Table 5.7: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.7 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python imple-

mentation takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.8: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
`accpipe_demo`	2.48	1.79	same accuracy, 0.9260
`parallel_accpipe`, run 1	6.83	5.88	outputs match
`parallel_accpipe`, run 2	6.83	0.08	outputs match
`code_parallelizer`, run 1	5.81	0.014	outputs match
`code_parallelizer`, run 2	6.05	0.015	outputs match
`save_load`, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
`save_load`, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.8 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.9: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
`accpipe_demo`	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
`parallel_accpipe`	isolated cache, run 1	6.66	6.00	outputs match
`parallel_accpipe`	isolated cache, run 2	6.58	0.09	outputs match
`code_parallelizer`	isolated cache, run 1	5.59	1.04	outputs match
`code_parallelizer`	isolated cache, run 2	5.61	1.05	outputs match
`save_load`	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.10: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
`accpipe_demo`	normal	–	6.53	graph example startup
`parallel_accpipe`	isolated cache, run 1	included	14.37	cold compile path
`parallel_accpipe`	isolated cache, run 2	cached	8.33	warm cache path
`code_parallelizer`	isolated cache, run 1	0.87	8.74	temporary executable
`code_parallelizer`	isolated cache, run 2	1.04	9.00	temporary executable
`save_load`	normal	–	2.71	persistence example

Tables 5.9 and 5.10 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table remains readable. The examples were run one by one in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.4.0.1 Auto Preprocessing Module

Most of the preprocessing functions were tested using unit tests. These tests were used to check validation rules, artifact saving,and feature transformation. Real datasets were also used to test the system in a more practical way. These datasets were collected from Kaggle for tabular, text, and image data.

How the Auto Preprocessing testing works The Auto Preprocessing testing was done as a practical full-flow test. The idea was to run preprocessing on real datasets,

then pass the processed output to a model. This means the test does not only check that preprocessing runs without errors, but also checks that the returned data can actually be used for training, validation, and prediction.

For tabular datasets, the test starts by loading each dataset with its target column and problem type. The preprocessing flow splits the data, fits the needed encoders and scalers, transforms the features and target, applies feature selection, and saves the preprocessing artifacts. After that, a machine learning model is trained on the processed training data and evaluated on the processed validation data. Classification datasets are evaluated using Macro F1-score, and regression datasets are evaluated using the R-squared score. The same datasets are also processed by a baseline automated cleaning tool, so the final scores can be compared and the Accelera output can be checked against an existing preprocessing approach.

For text datasets, the test checks that raw text can be converted into features that a classifier can understand. The preprocessing flow receives the text column and target column, cleans the text, tokenizes it, applies TF-IDF, encodes the labels, and returns processed training and validation data. A classifier is then trained on the transformed text features and evaluated using Macro F1-score. This confirms that the text preprocessing output is numeric, consistent, and usable for a normal machine learning model.

For image classification, the test checks the full image preparation flow. The preprocessing reads images from class folders, validates image files, skips corrupted images, creates class-to-label mappings, resizes the images, applies augmentation when enabled, and returns PyTorch DataLoaders. A neural network model is then trained using these loaders. The inference path is also checked by loading the saved model and applying the testing preprocessing to new images. This verifies that the saved image size and class mapping can be reused during testing, not only during training.

The main concept in all three tests is to check preprocessing through the next step in the machine learning pipeline. If the tabular arrays, TF-IDF matrices, labels, image tensors, or saved artifacts were wrong, the model training or evaluation step would fail or give invalid results.

Table 5.11: Datasets Used in Module Testing

Dataset Name	Shape	Dataset Type	Problem Type
startup_founder_burnout_2026	(50000, 29)	Tabular	Classification
healthy_diet_calorie_intake	(6000, 15)	Tabular	Classification
main gaming_addiction	(250, 49)	Tabular	Classification
student_exam_performance_dataset	(10000, 17)	Tabular	Classification
heart	(1025, 14)	Tabular	Classification
Dataset_spine	(310, 7)	Tabular	Classification
diabetes_dataset	(100000, 31)	Tabular	Classification
student_placement_synthetic	(100000, 18)	Tabular	Classification
Titanic-Dataset	(891, 12)	Tabular	Classification
customer_purchase_data	(1500, 9)	Tabular	Classification
global_ai_jobs	(90000, 35)	Tabular	Regression
CarPrice_Assignment	(205, 26)	Tabular	Regression
Housing	(545, 13)	Tabular	Regression
PetImages	(24998,)	Image	Classification
DailyDialog	(11327, 2)	Text	Classification
TestReviews	(4321, 2)	Text	Classification

Table 5.12: Models and Evaluation Metrics Used in Module Testing

Dataset Name	Model Used	Evaluation Metric	Accelera Result
startup_founder_burnout_2026	RandomForest	Macro F1-score	0.9999
healthy_diet_calorie_intake	RandomForest	Macro F1-score	1.0000
gaming_addiction	RandomForest	Macro F1-score	1.0000
student_exam_performance_dataset	RandomForest	Macro F1-score	0.8573
heart	RandomForest	Macro F1-score	0.7860
Dataset_spine	RandomForest	Macro F1-score	0.7840
diabetes_dataset	RandomForest	Macro F1-score	0.9985
student_placement_synthetic	RandomForest	Macro F1-score	0.9986
Titanic-Dataset	RandomForest	Macro F1-score	0.7906
customer_purchase_data	RandomForest	Macro F1-score	0.9385
global_ai_jobs	RandomForest	R-squared Score	0.9227
CarPrice_Assignment	RandomForest	R-squared Score	0.9562
Housing	RandomForest	R-squared Score	0.6037
PetImages	ResNet model	Accuracy	0.9700
DailyDialog	XGBClassifier	Macro F1-score	0.9000
TestReviews	XGBClassifier	Macro F1-score	0.6080

These results show that the preprocessing output can be used by different models after transformation. The tabular results were strong in many datasets, especially the larger classification datasets and the car price regression dataset. The lower scores on datasets such as TestReviews show that preprocessing alone does not solve every modeling problem. Some datasets still need better features, model tuning, or more suitable models.

5.4.0.2 AutoML Module

Experimental Objectives The AutoML experiments compare Accelera AutoML with Auto-sklearn, FLAML, and TPOT on supervised tabular classification and regression tasks. The evaluation focuses on two questions: whether the systems reach comparable predictive quality, and how much wall-clock time they require. Classification is evaluated mainly by accuracy. Regression is evaluated by R^2, root mean squared error (RMSE), and mean absolute error (MAE). Runtime is reported in seconds.

Benchmark Protocol The classification experiments were executed through the AutoMLBenchmark (AMLB) runner using the `flaml_paper_39` classification suite and the `flaml_paper_1h` constraint. The reported classification evaluation covers 12 datasets. The regression benchmark was built from the Penn Machine Learning Benchmarks (PMLB) repository and executed through AMLB using the `pmlb_regression` suite with the `pmlb_30m` constraint. The reported regression evaluation covers 10 datasets.

All systems were run through the same benchmark interface. The recorded setup used one fold and one allocated core per task. The regression time budget was 1,800 seconds per dataset, while the classification constraint allowed up to one hour.

Classification Results Table 5.13 summarizes the stored classification results. Auto-sklearn obtained the highest mean accuracy at 0.8842. Accelera reached a mean accuracy of 0.8791, a difference of about 0.0051, while recording the lowest mean duration at 1,201.7 seconds. Its mean runtime was approximately one third of the runtimes recorded for the other systems.

Table 5.13: AutoML classification benchmark summary

System	Mean accuracy	Mean time (s)	Accuracy wins	Speed wins
Auto-sklearn	0.884178	3615.71	3	3
FLAML	0.881427	3655.63	6	0
Accelera	0.879076	1201.72	6	15
TPOT	0.877654	3953.78	8	1

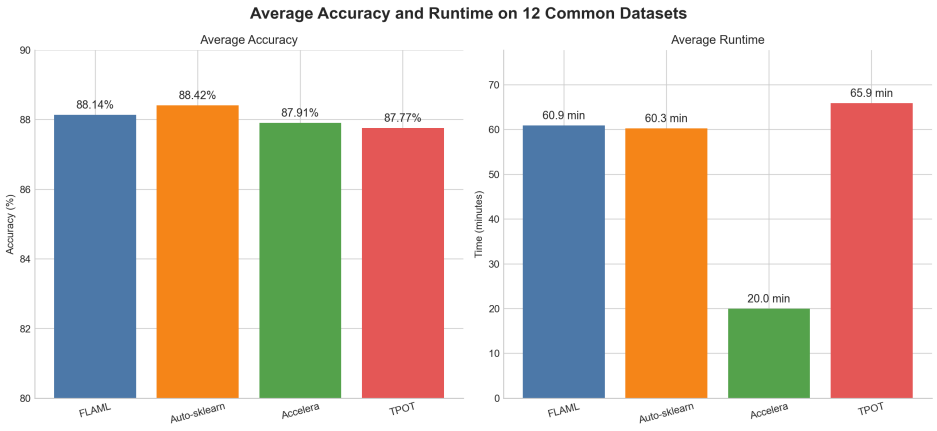

Figure 5.1: Mean classification accuracy and runtime for the evaluated AutoML systems

Regression Results Table 5.14 reports the regression summary over ten datasets. Auto-sklearn obtained the highest mean R^2 at 0.5609, followed closely by TPOT at 0.5592. Accelera reached 0.5294 and recorded the lowest mean runtime at 892.64

seconds, compared with 1,820–1,864 seconds for the competing systems. Accelera also achieved six speed wins, the most of any evaluated system.

Table 5.14: AutoML regression benchmark summary over ten datasets

System	Mean R^2	Median R^2	Mean time (s)	R^2 wins	RMSE wins	MAE wins	Speed wins
Auto-sklearn	0.560947	0.494362	1820.18	2	2	2	2
TPOT	0.559238	0.493144	1849.70	2	2	2	0
Accelera	0.529389	0.432577	892.64	2	2	2	6
FLAML	0.522743	0.497530	1863.85	4	4	4	2

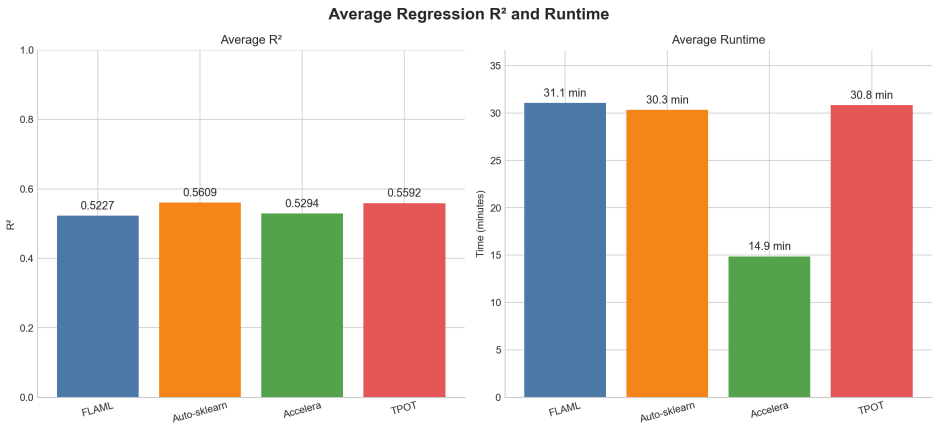

Figure 5.2: Mean regression R^2 and runtime for the evaluated AutoML systems

Threats to Validity The current results use a single benchmark fold, so they do not quantify variation across repeated splits or random seeds. Runtime is sensitive to hardware, operating-system load, framework startup overhead, and timeout handling. Aggregate means can also hide dataset-level failures or missing measurements. Therefore, the tables should be read as a summary of the stored runs rather than a universal ranking of AutoML systems.

5.4.0.3 Deployment Module

5.4.1 Experimental Setup and Test suite

The verification of the Deployment module is performed through a comprehensive test suite managed by the `pytest` framework. To ensure high test isolation and allow headless execution in CI/CD pipelines, the test suite leverages extensive mocking via the Python `monkeypatch` utility to decouple tests from system-level configurations, active network services (e.g., Heroku API, AWS EC2 hosts), and runtime dependency installations.

Tests are executed inside temporary filesystem workspaces generated dynamically using the `tmp_path` fixture. This ensures that the generated version control indices,

log files, and Docker deployment artifacts do not contaminate the primary workspace.

5.4.2 Subsystem Verification Details

The verification targets the six major components of the deployment module:

1. **Model Version Control (VCS) verification**: Validates the stability of the 7-character short SHA-1 hash calculation, confirming that identical timestamp-message pairs produce identical commit IDs. Tests ensure that `vcs.init()` initializes the experiments metadata directory structure and saves a clean registry template (`experiments.json`). Commit operations are verified to correctly copy model serialization binaries (e.g., `model.pkl`), snapshot active configurations, update the `HEAD` reference, and append metadata entries. Rollback capabilities are verified by deploying historical commits and confirming that the active runtime files are updated with the corresponding commit snapshots.

2. **Model Service Pipeline verification**: Tests check that the `ModelService` correctly parses configurations, loads active model estimators and preprocessors in the correct sequence, and performs inference. Mock tests confirm that preprocessing steps (such as scaling or encoding) are applied in order before predictions are generated.

3. **Runtime Schema Validation verification**: Validates the runtime behavior of input schema checking. Ephemeral Great Expectations suites are mocked to verify column-presence verification (identifying missing or unexpected fields) and value-bound checking. The data type auto-coercion system is verified by testing how it handles numeric conversions (casting strings to floats or integers) and parsing boolean representations (such as `"true"`, `"false"`, or numeric binary flags).

4. **Web Serving Endpoints verification**: Verifies that the FastAPI application correctly routes incoming HTTP requests. Tests evaluate the `/predict` and `/predict/csv` endpoints for JSON inputs and multi-row CSV file uploads. Endpoint testing checks that success responses return correct JSON structures with HTTP 200 codes, validation errors throw HTTP 422 codes, and server errors return HTTP 500 codes.

5. **Prediction Tracking and Telemetry verification**: Verifies the telemetry logging behavior. Tests ensure that when telemetry is disabled, no disk writes occur. When active, request parameters, model outputs, timestamps, and latency figures are verified to be written as valid JSON lines to the log file. Additionally, tests verify the telemetry reader's ability to aggregate counts (successful runs, error runs, total row counts) and report them dynamically.

6. **Orchestration CLI verification**: Validates port parsing limits (allowing ports between 1 and 65535, while rejecting negative numbers, non-numeric values, or out-of-bound ports). Tests check the preparation task (`prepare`) to verify that it creates a

clean staging directory, compiles a standard `Dockerfile` using a Python slim base, and builds a clean `requirements.txt` containing only runtime-specific dependencies.

5.4.3 Deployment Test Results

When executed in an environment with all core dependencies installed, all 51 test cases compile and pass successfully. Table 5.15 summarizes the verification results for the deployment module.

Table 5.15: Deployment Module Test Suite Verification Summary

Test Component File	Test Cases	Passed	Failed	Verified Feature
`vcs_test.py`	14	14	0	Registry snapshots, workspace ro
`modelservice_test.py`	6	6	0	Sequencing of preprocessor trans
`schema_validation_test.py`	8	8	0	Data coercion and column checki
`server_test.py`	9	9	0	FastAPI routing, telemetry integra
`tracking_test.py`	3	3	0	Telemetry logging and count aggr
`deployment_test.py`	11	11	0	Staging setup, Dockerfile compilin
Total	**51**	**51**	**0**	**End-to-End System Correctnes**

This successful validation confirms that the deployment engine operates as intended, offering robust models, automated runtime validation, telemetry logging, and repeatable container deployments without regression errors.

5.4.3.1 Accelera Pipeline Reporting Module

The pipeline reporting part was tested by generating reports from executed pipeline results. The test checks that the report can read the serialized graph file, create the graph image, detect branches, show the selected path, and display metric results in the correct format.

Different metric result types were checked, such as single numbers, arrays, dictionaries, strings, tuples, and plotted figures. The expected output was a `report.html` file with readable tables and figures. The graph image was also checked manually to make sure that selected nodes and normal nodes were shown using different colors.

5.4.3.2 Benchmark Platform Module

The Benchmark Platform module was evaluated using manual functional testing to check its main features from the user side. The testing focused on creating benchmark tasks, validating dataset links, submitting prediction files, calculating scores, and displaying the results in the leaderboard.

The leaderboard was tested by submitting different results and checking whether they were ordered correctly based on the selected evaluation metric. This included metrics where higher values are better and metrics where lower values are better.
The scoring process was also tested manually by using different target and prediction files. The platform was checked to make sure it rejects invalid submissions, such as files with different row counts or missing target columns. These tests showed that the benchmark platform can handle result submission, calculate scores, validate files, and display leaderboard results correctly.

5.4.4 Integration Testing

Integration testing was done by connecting the output of one part with another part. For Auto Preprocessing, the transformed data was passed to machine learning models to check that the output was usable. For pipeline reporting, the report was generated after graph serialization and metric calculation. For benchmarking, the backend called the Python scoring script and then saved the returned score in MongoDB.

These checks were important because a function can pass a unit test but still fail when used with the rest of the system.

5.4.5 Testing Schedule

The testing was done during implementation and after completing each module. First, small functions were tested. Then complete module flows were tested. After that, final outputs such as reports, saved files, and leaderboard results were checked manually. The final comparison with AutoCleanML was done after the tabular preprocessing flow became stable.

5.4.6 Comparative Results to Previous Work

5.4.6.1 Auto Preprocessing

Auto Preprocessing was compared with AutoCleanML to check the performance of the proposed preprocessing approach. The comparison was done using several tabular datasets collected from Kaggle. The same datasets and the same machine learning models were used for both tools. The comparison was applied only on tabular datasets because AutoCleanML mainly works with tabular data. For classification datasets, the Macro F1-score was used as the evaluation metric. For regression datasets, the R-squared score was used to measure how well the model predicts the target values. After preprocessing the datasets, the output data was tested using four machine learning models. For classification problems, Logistic Regression, Random Forest, Decision Tree, and KNN were used. For regression problems, Linear Regression, Random Forest, Decision Tree, and KNN were used. This helped test the preprocessing output with different model types, not only one model. The results were saved and plotted as comparison graphs between Accelera and AutoCleanML. These graphs show the

score of each tool on the selected datasets. The purpose of this comparison was to check whether Accelera can produce preprocessing results that are close to, or better than, an existing preprocessing tool.

Table 5.16: Classification comparison using Logistic Regression and Random Forest

Dataset	Logistic Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9999	0.9999
Healthy diet calorie intake	0.9917	0.9890	1.0000	1.0000
Gaming addiction	0.8937	0.8937	0.9646	1.0000
Student exam performance	0.8667	0.8657	0.8579	0.8573
Heart	0.8019	0.8007	0.7689	0.7860
Spine dataset	0.8691	0.7943	0.8360	0.7840
Diabetes dataset	0.9995	0.9985	0.9996	0.9985
Student placement synthetic	0.5657	0.5879	0.6241	0.9986
Titanic dataset	0.7734	0.7864	0.7906	0.7906
Customer purchase data	0.8484	0.8375	0.9386	0.9385

Table 5.17: Classification comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9824	0.9881
Healthy diet calorie intake	1.0000	1.0000	0.9379	0.9184
Gaming addiction	1.0000	1.0000	0.6905	0.8512
Student exam performance	0.7987	0.8063	0.8423	0.8338
Heart	0.7213	0.7704	0.7689	0.7860
Spine dataset	0.7686	0.7632	0.8239	0.8042
Diabetes dataset	0.9995	0.9972	0.9925	0.9980
Student placement synthetic	0.5827	0.9968	0.5995	0.8348
Titanic dataset	0.6979	0.7877	0.7203	0.7562
Customer purchase data	0.8486	0.8449	0.8544	0.8806

Table 5.18: Regression comparison using Linear Regression and Random Forest

Dataset	Linear Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.9022	0.7498	0.9288	0.9227
Car price assignment	0.8886	0.8943	0.9535	0.9562
Housing	0.6506	0.6482	0.6087	0.6037

Table 5.19: Regression comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.8640	0.8412	0.7586	0.7510
Car price assignment	0.9269	0.8825	0.7905	0.7478
Housing	0.3819	0.3634	0.5312	0.5336

From the classification and regression results, Accelera preprocessing achieved competitive performance compared to AutoCleanML. The comparison was tested using different types of models, including linear models, tree based models, Random Forest, and KNN. In many cases, the results of Accelera were very close to AutoCleanML, and in some cases Accelera achieved better scores.

These results show that the implemented preprocessing steps in Accelera are able to prepare data in a valid way for different machine learning techniques. The performance was not limited to one specific model type, but appeared across several models and datasets. This means that Accelera successfully reached results close to an existing automated preprocessing tool, while still using its own preprocessing approach.

However, the results also show that no preprocessing tool gives the best score in all cases. The final performance can still depend on the dataset, the problem type, and the model used after preprocessing.

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

6.3 Conclusions

Accelera provides a connected framework for preparing data, selecting models, building graph-based machine learning workflows, improving supported custom code, deploying trained models, and comparing results through a benchmark platform. The work shows that the project is not only a collection of separate tools, but a workflow where each module can support a different stage of the machine learning process.

The Auto Preprocessing module reduces repeated manual preparation work for tabular, text, and image datasets. It saves fitted artifacts so that testing data can be transformed in the same way as training data. The AutoML module adds automated model search under time and trial limits, which helps compare candidate models more systematically. Accelera Pipe represents workflows as graphs, so shared preprocessing, branches, predictions, and metrics can be organized more clearly. The Code Parallelizer adds a source-level optimization path for supported loop-heavy code while keeping conservative safety checks.

The Deployment module and Benchmark Platform complete the workflow by supporting model serving, version tracking, submission validation, scoring, and leaderboard comparison. The testing and experimental results show that the implemented modules can produce usable outputs, generate reports, evaluate models, and compare results. Overall, Accelera gives a practical base for a more organized and reusable machine learning workflow.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter , loop-analysis coverage

without weakening correctness checks, The preprocessing pipeline can be improved by adapting the preprocessing steps based on the selected model type and by adding techniques to handle imbalanced datasets. In the future, the benchmark platform can be improved by saving uploaded datasets on the server instead of using Google Drive links. It can also allow users to upload code files directly, instead of submitting only links.

Reference

[1] Matthieu Komorowski, Dominic C. Marshall, Justin D. Salciccioli, and Yves Crutain. “Exploratory data analysis.” In *Secondary Analysis of Electronic Health Records*, pages 185–203. Springer, 2016.

[2] Mohammad Hossin and Md Nasir Sulaiman. “A review on evaluation metrics for data classification evaluations.” *International Journal of Data Mining and Knowledge Management Process*, 5(2):1, 2015.

[3] Navin Sabharwal and Amit Agrawal. “Introduction to Natural Language Processing.” In *Hands-on Question Answering Systems with BERT: Applications in Neural Networks and Natural Language Processing*, pages 1–14. Springer, 2021.

[4] Koukaras Paraskevas and Tjortjis Christos. “Data preprocessing and feature engineering for data mining: Techniques, tools, and best practices.” *AI*, 6(10):257, 2025. MDPI AG.

[5] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms.” In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.

[6] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. “Efficient and robust automated machine learning.” In *Advances in Neural Information Processing Systems*, 2015.

[7] Randal S. Olson, Nathan Bartley, Ryan J. Urbanowicz, and Jason H. Moore. “Evaluation of a tree-based pipeline optimization tool for automating data science.” In *Applications of Evolutionary Computation*, 2016.

[8] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. “FLAML: A fast and lightweight AutoML library.” In *Proceedings of Machine Learning and Systems*, 2021.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] T. Kadosh, N. Hasabnis, P. Soundararajan, V. A. Vo, M. Capota, N. Ahmed, Y. Pinter, and G. Oren, “OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation,” arXiv preprint arXiv:2409.14771, 2024.

[11] Nick Erickson et al. “AutoGluon-Tabular: Robust and accurate AutoML for structured data.” *arXiv preprint arXiv:2003.06505*, 2020.

[12] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Sequential model-based optimization for general algorithm configuration.” In *Learning and Intelligent Optimization*, 2011.

[13] James Bergstra and Yoshua Bengio. “Random search for hyper-parameter optimization.” *Journal of Machine Learning Research*, 13:281–305, 2012.

[14] David H. Wolpert. “Stacked generalization.” *Neural Networks*, 5(2):241–259, 1992.

Chapter A: Development Platforms and Tools

A.1 Hardware Platform

A.2 Software Platform

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, Pandas, Matplotlib, PyTorch, Scikit-learn, Node.js, React, and MongoDB. The Auto Preprocessing module depends mainly on Python data-science libraries. The Pipeline Reporting module uses Python, HTML report generation, Matplotlib, Pandas tables, and Graphviz for graph visualization. The Benchmark Platform uses a Node.js and Express backend, MongoDB for storing benchmarks and submissions, React for the frontend, and Python scripts for dataset validation and metric scoring. The available experiments include Linux and Windows execution paths.

Chapter B: Use Cases

B.1 Auto Preprocessing Use Cases

The user can use Auto Preprocessing when they have a dataset and want to prepare it before training a model. For tabular data, the module can clean missing values, encode categorical columns, scale numerical columns, select features, save artifacts, and generate a report. For text data, it can clean sentences, tokenize them, apply TF-IDF, encode labels, and generate a report. For image data, it can validate images, map class folders to labels, resize images, apply augmentation, and create DataLoaders.

B.2 Pipeline Reporting Use Cases

The user can use the pipeline report after running an Accelera pipeline. The report helps show the graph structure, branches, selected path, and metric results. This is useful when the pipeline contains more than one branch and the user wants to understand which path was selected.

B.3 Benchmark Platform Use Cases

The benchmark platform can be used when different users or models need to be compared on the same task. A benchmark creator defines the dataset, test file, target column, and metric. Users then submit prediction files, and the platform calculates the score and displays the leaderboard.

Chapter C: User Guide

C.1 Using Tabular Auto Preprocessing

To use tabular preprocessing, the user first loads the dataset as a Pandas DataFrame. Then the user gives the target column, problem type, and output folder. The output folder is important because the module saves the fitted preprocessing objects and the HTML report inside it.

Code Example C.1: Calling tabular training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_training_preprocessing import (
    ClassicalTrainingPreprocessing,
)

df = pd.read_csv("dataset.csv")

preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="target",
    problem_type="classification",
    folder_path="outputs/tabular_run",
    val_size=0.2,
    random_state=42,
    is_report=True,
)

X_train, y_train, X_val, y_val = preprocessor.common_preprocessing()
```

For regression, the user only changes the problem type:

Code Example C.2: Changing tabular preprocessing to regression

```
preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="price",
    problem_type="regression",
    folder_path="outputs/regression_run",
)
```

After training preprocessing, the same folder should be used for test data. This makes the test data use the saved training transformers instead of fitting new ones.

Code Example C.3: Calling tabular testing preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_testing_preprocessing import (
    ClassicalTestingPreprocessing,
)
```

```
test_df = pd.read_csv("test.csv")

test_preprocessor = ClassicalTestingPreprocessing(
    df=test_df,
    folder_path="outputs/tabular_run",
)

X_test, y_test = test_preprocessor.common_preprocessing()
```

C.2 Using Text Auto Preprocessing

For text preprocessing, the user provides the text column and the target column. The text column must contain text values, and the target is used for classification.

Code Example C.4: Calling text training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.text_training_preprocessing import (
    TextTrainingPreprocessing,
)

df = pd.read_csv("reviews.csv")

text_preprocessor = TextTrainingPreprocessing(
    df=df,
    target_col="label",
    text_col="review",
    folder_path="outputs/text_run",
    tfidf_max_features=500,
    is_report=True,
)

X_train, y_train, X_val, y_val = text_preprocessor.common_preprocessing()
```

For testing text data, the user again passes the same folder path. If the test data does not contain the target column, the class returns the transformed text features and `None` for the labels.

Code Example C.5: Calling text testing preprocessing

```
from accelera.src.auto_preprocessing.core.text_testing_preprocesing import (
    TextTestingPreprocessing,
)

test_text = pd.read_csv("reviews_test.csv")

text_test_preprocessor = TextTestingPreprocessing(
    df=test_text,
    folder_path="outputs/text_run",
```

```
)

X_test, y_test = text_test_preprocessor.common_preprocessing()
```

C.3 Using Image Auto Preprocessing

For image preprocessing, the dataset should be arranged in class folders. For example:

```
Training/
  Cats/
  Dogs/
```

The user passes the training folder and the output folder. If a validation folder already exists, it can be passed also. If not, the module can split the training folder.

Code Example C.6: Calling image training preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_training_preprocessing
    import (
    ClassificationImageTrainingPreprocessing,
)

image_preprocessor = ClassificationImageTrainingPreprocessing(
    training_folder_images="data/PetImages/Training",
    folder_path="outputs/image_run",
    split_training=True,
    val_size=0.2,
    images_size=(224, 224),
    batch_size=16,
    augment=True,
)

train_loader, val_loader = image_preprocessor.common_preprocessing()
```

For image testing, the user provides a list of image paths. If class names are available, they can be passed too. If not, the loader can still be used for prediction.

Code Example C.7: Calling image testing preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_testing_preprocessing
    import (
    ClassificationImageTestingPreprocessing,
)

image_paths = ["test/cat1.jpg", "test/dog1.jpg"]
class_names = ["Cats", "Dogs"]

image_test_preprocessor = ClassificationImageTestingPreprocessing(
    image_paths=image_paths,
```

```
    image_class_names=class_names,
    folder_path="outputs/image_run",
    batch_size=4,
)

test_loader, invalid_images = image_test_preprocessor.common_preprocessing()
```

C.4 Reading Auto Preprocessing Reports

After running preprocessing with `is_report=True`, the user opens the generated `report.html` file from the output folder. The report shows the dataset overview, missing values, duplicate rows, dropped columns, graphs, preprocessing decisions, selected features, and processed data samples.

C.5 Using Pipeline Reports

To generate a pipeline report, the pipeline graph should be serialized first. Then the user passes the XML graph path, output folder, and metric results to `GraphReport`.

Code Example C.8: Calling the pipeline graph report

```
from accelera.src.accelera_pipe.wrappers.graph_report import GraphReport

report = GraphReport(
    folderpath="outputs/pipeline_report",
    xmlpath="outputs/pipeline_graph.xml",
    results=metric_results,
)

report.execute()
```

The output is a `report.html` file. It contains the graph image, branch tables, selected branch, and metric summary.

C.6 Using the Benchmark Platform

The benchmark platform is used from the web interface. The benchmark creator enters the title, problem type, target column, dataset link, test-set link, target-label link, and evaluation metric. A user submits a prediction file link and repository link. The backend calls the Python scoring script, saves the score, and displays the result in the leaderboard.

The prediction file must contain the target column name expected by the benchmark. Also, the number of rows in the prediction file must match the number of rows in the target-label file.

Chapter D: Code Documentation

Chapter E: Feasibility Study